

从无到有造 MQ：用 Java 构建一个轻量级消息中间件

内容精要

第一章 你好，消息中间件

内容：解释消息中间件的概念、应用场景、核心模型（生产者、Broker、消费者、主题、队列）。对比常见的 MQ（Kafka、RocketMQ、RabbitMQ、QMQ），说明自研 MQ 的学习意义。确立本书目标：从零实现一个具备持久化、延时消息、标签过滤和监控等特性的 Java MQ。

本章小结：本章我们了解了 MQ 的核心价值与基本概念，并定下清晰的技术目标——用一个完整的实践项目贯穿整本书，你将在后续章节中亲手实现每一个核心特性。

第二章 搭起骨架：项目结构与通信协议

内容：使用 Maven 创建多模块项目（common, broker, client, example）。引入 Netty 依赖，定义消息基类（Message）、命令枚举（Command）。基于 LengthFieldBasedFrameDecoder 和自定义编解码器解决 TCP 粘包问题，采用 JSON 序列化，实现一个最简单的“发消息-打印-响应”流程，让 Broker 和 Client 跑通。

本章小结：我们用 Netty 搭建了消息中间件的通信骨架，虽然只是简单的“发-回”原型，但它解决了网络传输、协议解析等关键问题，是后续所有高级功能的基石。

第三章 把消息存下来：顺序写文件日志

内容：介绍 CommitLog 设计思想。实现 MessageLog 类，支持按物理偏移量顺序追加写入，并按偏移量随机读取。设计消费者索引文件（ConsumeIndexManager），为每个 Topic+ConsumerGroup 存储消息偏移量。引入“消费进度”文件，实现简单的消费和 ACK 确认流程。

本章小结：顺序写磁盘是高性能 MQ 的核心秘诀，现在我们的 Broker 能够可靠地存储消息并记录每个消费者的进度，消息再也不会因为进程重启而丢失。

第四章 让消费者即时收到：长轮询

内容：分析常规轮询的弊端，提出长轮询方案。在 Broker 中实现挂起拉取请求、设置超时、消息到达后唤醒等待者的逻辑，将端到端延迟降至毫秒级，同时减少无效网络交互。

本章小结：长轮询让我们的 MQ 既有 Pull 模式的灵活性，又有 Push 模式的实时性，是生产环境必备的优化手段，实现起来也相当直观。

第五章 跨时间的投递：任意延时消息

内容：介绍 QMQ 的两级时间轮模型。实现磁盘时间轮（按小时分片持久化）和内存时间轮（Netty HashedWheelTimer）。设计 DelayMessageScheduler，重启时从磁盘恢复未到期消息。实现了从“10 秒后投递”到“2 年后投递”的灵活延时能力。

本章小结：延时消息是 QMQ 最亮眼的功能，通过磁盘+内存的两级调度，既保证长时间延时的持久化，又实现了到期时的精准投递，这个方案具有很强的工程实用性。

第六章 消息分类器：标签过滤

内容：扩展消息结构，增加 Tags 字段。生产者在发送时指定标签，消费者在拉取时声明订阅标签。服务端在拉取阶段进行匹配，不匹配的消息自动跳过，实现基于标签的服务端过滤，减少无效传输。

本章小结：标签过滤为 Topic 提供了更细粒度的分流能力，我们的实现与主流 MQ 的 Tag 过滤机制完全一致，代码简洁却高效。

第七章 能看得到的系统：Prometheus 监控

内容：引入 Micrometer 和 Prometheus Registry。实现独立的 Metrics HTTP 端点，暴露生产/消费计数、消费积压量、活跃连接数等指标。配置 Prometheus 和 Grafana，展示可视化看板，让系统运行状态一目了然。

本章小结：没有监控的 MQ 就像没有仪表盘的汽车，我们通过几行代码就为系统注入了可观测性，这对运维至关重要。

第八章 提升吞吐量：批量发送与拉取

内容：修改协议支持一次传输多条消息。生产者新增 sendBatch 方法，消费者支持 maxMessages 参数。服务端在拉取循环中一次性收集多条匹配消息返回，大幅减少网络往返开销，提升小消息的吞吐能力。

本章小结：批量处理是消息系统优化的终极武器之一，我们的实现不仅提升了吞吐量，还保持了标签过滤和长轮询等特性的兼容性。

第九章 消息的最终归宿：重试与死信队列

内容：介绍消息投递失败的处理策略。在消费者索引管理中增加投递次数记录，达到重试上限后自动将消息写入 DLQ-原 topic 死信主题。通过 Prometheus 暴露死信计数，帮助开发者及时发现坏消息。

本章小结：没有完美的业务代码，只有健壮的消息中间件。死信队列给了坏消息一个“临终关怀”，避免因单条消息反复失败而拖垮整个消费者。

第十章 优雅退场与重生：关闭与崩溃恢复

内容：注册 JVM shutdown hook，实现 Broker 的优雅关闭（停止接受新连接、等待请求处理完、关闭文件通道）。在 MessageLog 启动时自动修复因崩溃导致的文件尾部

损坏（截断不完整消息），保证系统迅速恢复服务。

本章小结：意外总是难免，但我们要做到“优雅地死，干净地活”。本章的两个机制让 MQ 能在重启和崩溃后保持数据一致性，大大提升了生产环境的稳定性。

第十一章 客户端的自我修养：断线重连

内容：分析 Broker 重启或网络故障对客户端的影响。在 MQClient 中实现连接断开检测与自动重连（带退避策略），并在 Consumer 和 Producer 的高层 API 中加入重试逻辑，使客户端能在 Broker 恢复后无缝继续工作。

本章小结：客户端不再是脆弱的“玻璃娃娃”，它能自动适应网络波动，这为后续集群化、滚动升级打下了坚实的基础。

第十二章 从任意时间开始：按时间消费

内容：为消息添加时间戳，构建时间索引。消费者可以指定起始消费时间点，Broker 通过二分查找快速定位到对应偏移量，并跳过更早的消息，实现“从 10 分钟前开始消费”这样的业务需求。

本章小结：时间维度的消费控制是很多场景的刚需，我们的实现没有引入复杂的存储引擎，只是巧妙地利用了顺序写入的特性，简单而有效。

第十三章 管理员的工具箱：管理 API 与部署

内容：扩展 Metrics HTTP 服务，提供 /admin/topics、/admin/consumers、/admin/stats 等运维接口，返回 JSON 数据。编写 Dockerfile 和 docker-compose.yml，将 Broker 容器化部署，并整合 Prometheus 监控栈。

本章小结：一个成熟的消息中间件离不开易用的管理界面和便捷的部署方式，现在我们的系统已经可以打包成 Docker 镜像，随时准备在任何地方运行。

第十四章 压测与优化：冲刺十万 TPS

内容：设计性能测试方案，使用多线程模拟生产者和消费者，测量吞吐量、延迟和系统资源消耗。分析瓶颈，讨论批量发送、异步落盘、零拷贝、切换序列化协议等优化方向，让读者理解 MQ 性能调优的真实路径。

本章小结：性能是一个系统工程，本章的测试数据和优化思路为后续进入更高量级的实际应用提供了参考。

第十五章 未来之路：从原型到集群

内容：总结本书已实现的功能，指出单机系统的局限性。展望集群化改造方向：元数据服务器、分区、主从复制、负载均衡、数据迁移等。介绍 QMQ 的真实架构演进，鼓励读者进一步探索。

本章小结：我们的 MQ 已经从一个玩具成长为功能完备的单机中间件，但真正的生产级系统还需要多机协作。本章留下的设计蓝图，正是你下一步挑战的起点。

第一章 你好，消息中间件

本章前序

假设你正在开发一个电商系统。用户下单后，需要做很多事情：扣减库存、生成订单、发送短信通知、记录日志、给用户增加积分、触发风控检查……如果把这些操作全部写在一个下单接口里，这个接口会变得臃肿不堪，响应时间极长。更可怕的是，一旦其中某个环节失败（比如积分服务挂了），整个下单流程都可能被拖垮。

再想象另一个场景：你的系统平时每秒只有几百个请求，但双十一零点，流量瞬间暴涨到每秒几万。如果你直接让所有请求都压向数据库，数据库很可能直接被打挂。

这些问题的本质，是系统间直接同步调用导致的耦合过紧、吞吐受限和脆弱性。有没有一种办法，能让各个服务“优雅地沟通”，既不互相阻塞，又能扛住流量冲击？答案是：在它们之间，加一个“信使”——**消息中间件**。

本章将带你走进消息中间件的世界，理解它的核心概念、适用场景，并纵览市面上主流的 MQ 产品。最后，我们会确立本书的终极目标：**从零开始，用 Java 亲手实现一个包含持久化、延时消息、标签过滤和监控等功能的轻量级消息中间件**。准备好了吗？让我们启程。

1.1 什么是消息中间件

消息中间件（Message-Oriented Middleware, MOM），你可以把它理解成一个“**智能邮局**”。

在现实世界中，小明要给小红写信，他不需要亲自跑到小红家门口把信塞进信箱，而是把信投进离自己最近的邮筒。邮局负责分拣、运输，最终将信准确地送到小红手里。在这个过程中：

- 小明和小红不需要同时在场（异步）；
- 小明不需要知道小红的具体地址，他只需要写好收件人（解耦）；
- 即使小红出差了，邮局也可以暂时保管信件，等她回来再投递（缓冲/削峰）。

把这个模型搬到软件世界里：

- **小明就是生产者（Producer）**，负责发送消息。
- **小红就是消费者（Consumer）**，负责接收和处理消息。
- **邮局就是消息代理（Broker）**，负责接收、存储和转发消息。
- 信件上写的“小红”就是**主题（Topic）**或**队列（Queue）**，用于标识消息应该发

给谁。

因此，消息中间件本质上是一个**跨进程、跨机器的异步通信中间层**。它允许生产者将消息发送到 Broker，消费者在合适的时候从 Broker 拉取（或 Broker 主动推送）消息进行处理，从而将生产者和消费者在时间、空间和代码层面完全解耦。

1.1.1 核心模型

让我们再深入一点。主流的消息中间件通常围绕以下几个核心概念构建：

- **消息 (Message)**：传递的数据单元，通常包含消息体（业务数据）和一些元数据（如标签、键值对）。
- **生产者 (Producer)**：负责创建并发送消息的应用程序。
- **消费者 (Consumer)**：负责接收并处理消息的应用程序。
- **主题 (Topic)**：消息的逻辑分类，类似于“新闻频道”。生产者将消息发到某个 Topic，消费者订阅该 Topic 来接收消息。这是发布/订阅 (Pub/Sub) 模式的基础。
- **队列 (Queue)**：在点对点 (P2P) 模式下，一条消息通常只会被一个消费者消费，消费后即被移除。但在更现代的模型中（如 Kafka 的消费者组），多个消费者可以共享一个 Topic，但每条消息只被组内一个消费者处理。
- **Broker**：消息中间件的服务端，负责接收、持久化、路由和投递消息。它是整个系统的中枢。

1.1.2 两种主要消息模式

1. 点对点模式 (P2P)

生产者将消息发送到一个队列中，一个或多个消费者可以监听该队列，但**一条消息只会被一个消费者消费**，消费后即被移除。这类似于多个工人从同一个任务池中领取任务。

2. 发布/订阅模式 (Pub/Sub)

生产者将消息发布到某个 Topic，多个消费者可以订阅该 Topic，**每个订阅者都会收到一条完整的消息副本**。这类似于报纸订阅，报社印一份报纸，所有订阅者都能收到。

现代 MQ 通常同时支持这两种模式，甚至允许在消费者组内进行负载均衡（如 Kafka 的 Consumer Group）。

1.2 消息中间件的应用场景

消息中间件之所以成为分布式系统的必备组件，是因为它优雅地解决了几类通用问题：

1.2.1 系统解耦

在一个复杂的分布式系统中，服务之间如果直接通过 RPC 调用，就会形成“网状”的依赖关系。一旦某个服务变更接口或宕机，调用方就会受到直接影响。通过引入 MQ，生产者只负责将消息发送到 Broker，不关心谁来消费；消费者也只从 Broker 获取消息，不关心消息来自哪里。**双方都只依赖 Broker 和消息格式**，真正实现了面向接口编程。

案例：用户注册后，需要发送欢迎邮件、初始化账户、赠送新人大礼包。注册服务只需发一条“用户注册成功”的消息到 Broker，其他服务各自订阅，互不干扰。

1.2.2 异步处理

有些业务操作（如发送短信、记录日志、生成报表）不需要立即完成，将它们放入消息队列异步执行，可以**大幅缩短主流程的响应时间**，提升用户体验。

案例：订单支付成功后，主流程只需确认支付状态并返回用户，至于后续的发票生成、物流通知等操作，全部通过消息异步触发。

1.2.3 削峰填谷

在秒杀、抢购等高并发场景下，瞬间流量可能是平时的几十倍。如果直接将请求压向后端服务，系统可能因超负荷而崩溃。MQ 就像一个**蓄水池**，前端涌入的请求快速写入 MQ，后端服务按照自身处理能力匀速消费，从而平稳度过流量高峰。

案例：双十一零点，海量订单涌入，订单服务先将请求快速写入 MQ，后续的库存扣减、支付处理等服务按自己的节奏处理，系统不会过载。

1.2.4 数据分发与流处理

消息中间件也是大数据领域的基础设施。日志、用户行为、传感器数据等可以被源源不断地写入 MQ，然后被多个下游系统（如实时计算引擎、离线分析平台、搜索引擎）同时消费，形成数据管道。

案例：App 的点击流日志被写入 Kafka，实时流处理框架 Flink 进行实时大屏展示，同时批量任务将数据导入 Hadoop 做日活分析。

1.3 主流消息中间件巡礼

在决定“造轮子”之前，我们先看看市面上那些强大的“轮子”长什么样，理解它们的设计取舍，才能更好地吸收精华。

1.3.1 RabbitMQ

用 Erlang 语言编写，基于 AMQP 协议，是**企业级消息队列的标杆**。它提供极其灵活的路由能力（Exchange、Binding、Queue 的组合），支持死信、延迟队列等高级特性，稳定性极高。但吞吐量相对较低（万级 TPS），在互联网海量数据场景下略显吃力。

1.3.2 Kafka

LinkedIn 开源的分布式流平台，使用 Scala/Java 编写。它以**超高的吞吐量（百万级 TPS）和持久化**著称，采用顺序写磁盘和零拷贝技术，是日志、流处理领域的王者。Kafka 的模型是 Topic-Partition，消费进度由消费者自己管理。但它对延时消息的支持较弱，功能相对单一。

1.3.3 RocketMQ

阿里巴巴开源的第三代分布式消息中间件，使用 Java 编写，源于阿里多年的电商实践。它借鉴了 Kafka 的高性能持久化设计，又吸收了 RabbitMQ 的丰富特性（如事务消息、延时消息、死信队列），在**低延迟、高可靠和功能丰富之间取得了很好的平衡**。经历过双十一的考验，是金融级互联网应用的优秀选择。

1.3.4 QMQ

去哪儿网自研的消息中间件，也是本书灵感的重要来源。QMQ 的亮点在于**数据库级别的事务消息和超长延时消息**。事务消息通过将业务数据库与消息存储绑定，保证消息的绝对一致性；延时消息通过两级时间轮，支持长达数年的任意时间延迟，非常适合旅游出行的提醒场景（如航班起飞前 2 小时通知）。

1.3.5 比较与选择

特性	RabbitMQ	Kafka	RocketMQ	QMQ
吞吐量	万级	百万级	百万级	十万级
延迟	微秒级	毫秒级	毫秒级	毫秒级
事务消息	不支持	不支持	支持	支持（DB 级别）
延时消息	有限支持	不支持	固定等级	任意时长，两级时间轮
语言	Erlang	Scala/Java	Java	Java

你会发现，每个 MQ 都有自己最闪光的设计。学习它们，并结合自己的业务需求去思考，正是我们自研 MQ 的初衷。

1.4 为什么要自研消息中间件

或许你会问：既然有这么多成熟的开源 MQ，为什么还要自己写一个？

1.4.1 知其然，更知其所以然

使用 MQ 很简单，引入依赖，几行代码就能收发消息。但一旦出现消息丢失、延迟飙升、消费堆积等问题，你对底层原理的理解程度就决定了你排查问题的速度。亲手实现一个 MQ，你会深入到序列化、存储、索引、网络通信的每一个细节，**彻底吃透消息队列的底层机制**。

1.4.2 掌握通用的系统设计范式

消息中间件浓缩了大量分布式系统的设计精髓：顺序写与随机读的取舍、零拷贝、时间轮调度、滑动窗口、主从复制、一致性协议……这些技术在你设计数据库、缓存、网关、实时计算等系统时，同样会大放异彩。

1.4.3 量身定做，追求极致

开源产品为了通用性，往往牺牲了一些特定场景下的极致力。如果你需要一个极轻量、可嵌入、或者支持独特业务逻辑（比如我们后面要实现的任意时间延迟）的 MQ，自研就是最佳路径。许多大厂的自研 MQ 都是这么诞生的。

1.4.4 学习的乐趣与成就感

没有什么能比“我写了一个 MQ”更让一个后端工程师感到自豪的了。从一行行代码中构建出一个能稳定运行的系统，这种创造的快乐，是单纯的 API 调用无法比拟的。

1.5 本书的目标与路线图

基于以上思考，本书的目标不是再造一个 Kafka 或 RocketMQ，而是**用最精炼的代码，实现一个功能完备、可直接运行的轻量级消息中间件**，并在这个过程中，逐一解释每个关键设计的原理和取舍。

我们将实现一个名为 **min-mq** 的 Java 项目，它具备以下特性：

- **核心功能**：消息的发布与订阅、多消费者组独立消费、消息持久化、消费进度管理。
- **高级特性**：任意时长的延时消息、基于 Tag 的服务端过滤、批量发送与拉取、消息重试与死信队列。

- **运维与监控**：Prometheus 指标暴露、管理 API、优雅关闭与崩溃恢复、客户端断线重连。
- **生产环境支持**：分段日志存储、按时间消费、Docker 容器化部署。

全书共十五章，我们将从搭建项目骨架开始，一步步添加持久化、长轮询、延时消息、标签过滤、监控、批量处理、死信、重连、时间消费、管理接口和压测优化，最终呈现一个完整的、可以持续演进的消息系统。

1.6 本章小结

本章，我们通过“智能邮局”的比喻，理解了消息中间件的核心价值：**解耦、异步、削峰**。我们辨析了 Topic、Queue、Producer、Consumer 等基本概念，并对比了 RabbitMQ、Kafka、RocketMQ 和 QMQ 的设计哲学。最后，我们明确了为什么要亲手实现一个 MQ，以及本书将要达成的目标。

从下一章开始，我们将正式动工，用代码为“信使”注入生命。系好安全带，让我们出发吧！

第二章 搭起骨架：项目结构与通信协议

本章前序

在第一章中，我们明白了消息中间件的核心价值，也确立了要亲自动手实现一个具备多种特性的轻量级 MQ。现在，是时候放下理论，拿起键盘了。

任何一个分布式系统，都离不开最基础的两件事：**组织代码**和**让不同进程通信**。如果把最终完成的 MQ 比作一栋大楼，那么本章要做的，就是打下地基、架起钢筋骨架。我们会创建一个 Maven 多模块项目，引入 Netty 网络框架，并设计一套简单的二进制通信协议。最终，我们将实现一个最简化的消息交互流程：客户端发送一条消息，服务端接收并打印，然后返回一个确认。

别看这个流程简单，它包含了网络编程中至关重要的几个环节：序列化、协议解析、粘包处理、请求响应匹配。这些技术不仅是 MQ 的基础，也是所有分布式系统的基础。让我们一起，为“信使”注入第一行代码。

2.1 项目工程结构：多模块的优雅

在实际开发中，我们不会把所有代码塞进一个模块。那样会臃肿不堪，而且客户端和服务端耦合在一起，无法独立部署。因此，我们将采用 Maven 的多模块（Multi-module）结构，将项目拆分为四个子模块：

text

min-mq/

—— pom.xml	# 父 POM，管理依赖版本
—— min-mq-common/	# 公共模块：协议定义、消息编解码
—— min-mq-broker/	# 服务端：Broker 逻辑
—— min-mq-client/	# 客户端：Producer、Consumer API
—— min-mq-example/	# 示例：启动类和测试代码

- **min-mq-common**：存放协议相关的类，比如消息体、命令枚举、编解码器。服务端和客户端都要依赖它。
- **min-mq-broker**：Broker 的实现，包括网络监听、消息处理等。它依赖 min-mq-common。
- **min-mq-client**：封装了生产者和消费者的 API，业务系统只需引入这个模块即可与 Broker 通信。它同样依赖 min-mq-common。

- **min-mq-example**: 用于快速启动 Broker 和运行客户端示例，依赖 min-mq-broker 和 min-mq-client。

每个模块都有自己的 pom.xml，父 POM 统一管理依赖版本，如 Netty、Jackson 等。这种结构清晰、职责明确，也方便后续扩展和打包。

2.2 通信协议设计：让字节说话

网络通信的本质是字节流的传输。两个进程要想“听懂”对方在说什么，必须先约定好一套**协议**。我们的协议设计将围绕几个核心问题：如何标识一条消息的类型？如何携带数据？如何把请求和响应对应起来？

2.2.1 命令枚举

我们定义了 Command 枚举，代表不同的操作类型：

```
java

public enum Command {

    SEND,          // 发送消息

    PULL,          // 拉取消息

    ACK,           // 确认消费

    RESPONSE       // 通用响应

}
```

这四者覆盖了基本的通信模式：生产者发送 SEND，消费者发送 PULL 和 ACK，服务端对任何请求都回复一个 RESPONSE。

2.2.2 消息结构

我们设计了一个 Message 类，它将被序列化为字节流在线路上传输：

```
java

public class Message {

    private String command;          // 命令类型

    private String requestId;        // 请求唯一标识，用于匹配响应

    private String topic;            // 主题

    private String body;             // 消息体

}
```

```
private Map<String, String> headers = new HashMap<>(); // 扩展头部

// 后续章节还会增加 group、tags、delayMs 等字段

}
```

requestId 是核心设计：客户端每发一个请求，都会生成一个 UUID 作为 ID，并把一个 CompletableFuture 放入一个 Map；服务端在响应时原封不动带回这个 ID，客户端收到响应后取出对应的 Future 并完成它，从而实现异步请求-响应匹配。这种模式在异步网络编程中极为常见。

2.2.3 序列化方式：JSON

为什么选择 JSON？对于学习型项目，**可读性和调试便利性**是第一位的。Jackson 库可以快速将 Java 对象转为字节数组，即使以后换成 Protobuf 也只需要修改编解码器。在如今的阶段，JSON 完全够用。

2.3 引入 Netty：高性能网络通信的基石

Java 原生 NIO 的 API 复杂且容易出错，直接使用它处理高并发连接很容易写出 bug。Netty 是一个异步事件驱动的网络应用框架，它封装了底层 NIO 的复杂性，提供了简单易用的 Channel 抽象、灵活的 Pipeline 机制，以及丰富的协议编解码器。

在我们的项目中，Netty 同时扮演服务端和客户端的角色。服务端使用 ServerBootstrap 监听端口，客户端使用 Bootstrap 发起连接。两者都共享相同的 Channel 流水线配置。

2.3.1 服务端启动

BrokerServer 的核心代码如下：

```
java

EventLoopGroup bossGroup = new NioEventLoopGroup(1);

EventLoopGroup workerGroup = new NioEventLoopGroup();

ServerBootstrap b = new ServerBootstrap();

b.group(bossGroup, workerGroup)

  .channel(NioServerSocketChannel.class)

  .childHandler(new ChannelInitializer<SocketChannel>() {

    @Override
```

```

protected void initChannel(SocketChannel ch) {

    ch.pipeline()

        .addLast(new LengthFieldBasedFrameDecoder(1024*1024, 0, 4, 0, 4))

        .addLast(new LengthFieldPrepender(4))

        .addLast(new MessageCodec.Encoder())

        .addLast(new MessageCodec.Decoder())

        .addLast(new MessageHandler());

}

});

b.bind(port).sync();

```

bossGroup 负责接受连接，workerGroup 负责处理 I/O 事件。childHandler 定义了每条连接上的处理器链（Pipeline），数据流进或流出时，会依次经过这些处理器。

2.3.2 客户端连接

MQClient 的启动类似，但使用 Bootstrap 并指定服务端地址：

```

java

Bootstrap b = new Bootstrap();

b.group(group)

    .channel(NioSocketChannel.class)

    .handler(new ChannelInitializer<SocketChannel>() {

        @Override

        protected void initChannel(SocketChannel ch) {

            ch.pipeline()

                .addLast(new LengthFieldBasedFrameDecoder(1024*1024, 0, 4, 0, 4))

                .addLast(new LengthFieldPrepender(4))

                .addLast(new MessageCodec.Encoder())

                .addLast(new MessageCodec.Decoder())

                .addLast(new SimpleChannelInboundHandler<Message>() { ... });

```

```
    }  
    });  
b.connect(host, port).sync();
```

客户端的 Pipeline 和服务端基本对称，差异仅在于最后的业务处理器不同。

2.4 粘包/拆包与编解码：让字节流乖乖听话

TCP 是一个“流”协议，它没有消息边界。如果你连续发送两条消息，它们可能会粘在一起到达；或者一条消息被拆成两个 TCP 包送达。如果不加处理，接收端将无法正确解析出完整的一条条消息。

Netty 提供了开箱即用的解决方案：

LengthFieldBasedFrameDecoder 和 LengthFieldPrepender。

- LengthFieldPrepender：在写入数据前，在消息最前面加上 4 个字节的长度字段。
- LengthFieldBasedFrameDecoder：读取数据时，先解析这 4 个字节的长度，然后读取指定长度的字节，从而正确分割出一条完整消息。

我们在 Pipeline 的首部加入这两个处理器，就解决了粘包/拆包问题。

接下来是**编解码器**。MessageCodec 内部定义了两个静态类：Encoder（继承 MessageToByteEncoder<Message>）和 Decoder（继承 ByteToMessageDecoder）。它们负责将 Message 对象与字节数组互相转换：

```
java  
  
public static class Encoder extends MessageToByteEncoder<Message> {  
    @Override  
    protected void encode(ChannelHandlerContext ctx, Message msg, ByteBuf out)  
        throws Exception {  
        byte[] bytes = MAPPER.writeValueAsBytes(msg);  
        out.writeBytes(bytes);  
    }  
}
```

```

public static class Decoder extends ByteToMessageDecoder {

    @Override

    protected void decode(ChannelHandlerContext ctx, ByteBuf in, List<Object> out)
    throws Exception {

        byte[] bytes = new byte[in.readableBytes()];

        in.readBytes(bytes);

        Message msg = MAPPER.readValue(bytes, Message.class);

        out.add(msg);

    }

}

```

因为前面已经有长度处理器确保 Decoder 收到的就是一条完整消息的字节，这里就可以直接反序列化了。

2.5 实现一个最简单的消息交互

万事俱备，我们来实现第一阶段的核心逻辑：**客户端发送一条消息，服务端打印并返回确认。**

2.5.1 服务端 MessageHandler

MessageHandler 是服务端业务处理的入口。它继承 SimpleChannelInboundHandler<Message>，在 channelRead0 中根据命令进行分发：

```
java
```

```
@Override
```

```
protected void channelRead0(ChannelHandlerContext ctx, Message msg) {
```

```
    Command cmd = msg.getCommand();
```

```
    switch (cmd) {
```

```
        case SEND:
```

```
            System.out.println("Received message: topic=" + msg.getTopic() + ",
body=" + msg.getBody());
```

```

        Message ack = new Message(Command.RESPONSE, msg.getTopic(),
"OK");

        ack.setRequestId(msg.getRequestId());

        ctx.writeAndFlush(ack);

        break;

    case PULL:

        // 暂时不做持久化，直接返回 null

        Message resp = new Message(Command.RESPONSE, msg.getTopic(),
null);

        resp.setRequestId(msg.getRequestId());

        ctx.writeAndFlush(resp);

        break;

    default:

        // 返回错误

    }
}

```

这里我们还没有存储消息，只是简单打印并返回“OK”。PULL 请求则返回空的 body，表示没有消息可消费。

2.5.2 客户端 Producer 和 Consumer

客户端方面，MQClient 封装了连接和发送。send 方法返回一个 `CompletableFuture<Message>`：

```

java

public CompletableFuture<Message> send(Message msg) {

    CompletableFuture<Message> future = new CompletableFuture<>();

    pendingRequests.put(msg.getRequestId(), future);

    channel.writeAndFlush(msg).addListener(writeFuture -> {

        if (!writeFuture.isSuccess()) {

            pendingRequests.remove(msg.getRequestId());

```

```
        future.completeExceptionally(writeFuture.cause());
    }

});

return future;
}
```

Producer 调用 `client.send(msg).get()` 同步等待响应，Consumer 则发起 PULL 请求并等待返回。

2.5.3 示例运行

在 `ExampleBrokerMain` 中启动 Broker，在 `ExampleClientMain` 中创建客户端、发送一条消息、拉取一次：

```
java
```

```
MQClient client = new MQClient("localhost", 8080);
```

```
client.connect();
```

```
Producer producer = new Producer(client);
```

```
producer.send("test-topic", "Hello QMQ!");
```

```
Consumer consumer = new Consumer(client, "test-topic");
```

```
Message msg = consumer.pull(); // 当前版本返回 null
```

运行后，Broker 的控制台会打印 `Received message: topic=test-topic, body=Hello QMQ!`，客户端打印 `No message available`。虽然简单，但整个通信链路已经跑通。

2.6 运行与验证

执行以下命令即可启动：

```
bash
```

```
# 终端 1: 启动 Broker
```

```
mvn -pl min-mq-example exec:java -
```

```
Dexec.mainClass="com.minmq.example.ExampleBrokerMain"
```

终端 2: 运行客户端

```
mvn -pl min-mq-example exec:java -  
Dexec.mainClass="com.minmq.example.ExampleClientMain"
```

观察 Broker 控制台的输出，确认消息已被接收。如果一切正常，恭喜你，一个最基本的 MQ 通信骨架已经成型！

2.7 本章小结

本章我们完成了从零开始搭建项目到实现基本通信的全过程。我们用 Maven 多模块组织代码，设计了简单的命令协议和消息结构，借助 Netty 和长度前缀帧解决了 TCP 粘包问题，并使用 JSON 进行序列化。最终，我们让客户端成功地将消息发送到了 Broker，并收到了响应。

虽然这个版本还不能存储消息，也不能实现复杂的消费逻辑，但它奠定了整个项目的通信基础。在接下来的章节中，我们将为 Broker 装上“记忆”——将消息持久化到磁盘，让重启不再丢失数据。

现在，你的“信使”已经会说话了，下一步，我们要教它记住说过的每一句话。

第三章 把消息存下来：顺序写文件日志

本章前序

在第二章，我们搭建了项目骨架，让客户端和服务端能通过网络“对话”，并能将消息打印在控制台上。但这充其量只是一个“传声筒”——一旦 Broker 进程重启，所有消息就烟消云散了。对于严肃的业务场景，这是不可接受的。订单、支付、库存变更等关键数据必须被可靠地存储下来，即使断电、重启也不能丢失。

那么，该怎么存？存到关系型数据库吗？MySQL 虽然可靠，但写入性能和消息队列的海量吞吐相比，如同用家用轿车拉货柜。存到内存？快是快了，但断电即丢。现代消息中间件几乎无一例外地选择了**本地文件系统**作为存储介质：将消息直接追加写入日志文件，利用磁盘的顺序写特性获得极高的吞吐量，同时又能像数据库一样支持随机读取。

本章我们将为 Broker 装上“硬盘记忆体”。你会看到，只需少量的核心类，就能实现一个高性能、可恢复的消息存储引擎。我们将依次完成：

- 设计 CommitLog（消息日志）的存储格式与读写逻辑。
- 实现消费者索引与消费进度的持久化。
- 将存储层接入到 Network Layer，让发送的消息真正落盘，让消费者能从上次的位置继续消费。

准备好，我们要开始写磁盘了。

3.1 CommitLog 的设计哲学：顺序追加，随机读取

想象一下，你有一个无限长的卷纸，你不断在纸的末端一行行写字。当你需要找某一行时，你必须从卷纸的开头一直找。如果卷纸很长，查找效率极低。但如果你有一个“目录”，记录了每一行的起始位置，你就能瞬间跳到那行去阅读。

消息中间件的存储正是这个道理。**顺序写入**是磁盘最快的操作之一——现代的 SSD 顺序写速度轻松达到数百 MB/s，远高于随机写。而**随机读取**虽然比顺序读慢，但只要我们有“目录”（即索引），就能精准跳到目标位置，一次读取通常只涉及几次 I/O，性能完全可接受。

我们的 CommitLog 采用一种极简格式：每条消息 = [4 字节长度头] + [JSON 序列化的消息数据]。长度头让我们知道这条消息占了多少字节，从而可以连续读取下一条。

消息数据本身被封装在一个 MessageEntry 对象中，包含 topic、body、tags 和 timestamp，方便后续解析。

3.1.1 核心写入逻辑

在 MessageLog.append() 方法中：

java

```
public synchronized long append(String topic, String body, String tags, long timestamp)
{
    // 1. 构造消息条目
    MessageEntry entry = new MessageEntry(topic, body, tags, timestamp);

    // 2. 序列化为 JSON 字节
    byte[] data = mapper.writeValueAsBytes(entry);

    int length = data.length;

    // 3. 写入 [4 字节长度][数据]
    ByteBuffer buf = ByteBuffer.allocate(4 + length);

    buf.putInt(length);

    buf.put(data);

    buf.flip();

    long offset = wrotePosition; // 全局物理偏移量

    channel.write(buf);

    wrotePosition += (4 + length);

    return offset;
}
```

注意两点：synchronized 保证了写操作的串行性，避免多线程竞争破坏文件内容；返回的 offset 是这条消息在文件中的**起始字节位置**，我们称之为**物理偏移量**。这个偏移量是后续索引和随机读取的关键。

3.1.2 随机读取

当需要读取某条消息时，我们传入物理偏移量，方法会先读 4 字节长度，再读指定长度的 JSON 数据并反序列化：

java

```
public MessageEntry readMessage(long offset) {
```

```
    ByteBuffer lenBuf = ByteBuffer.allocate(4);

    channel.read(lenBuf, offset);

    int length = lenBuf.getInt(0);

    ByteBuffer dataBuf = ByteBuffer.allocate(length);

    channel.read(dataBuf, offset + 4);

    return mapper.readValue(dataBuf.array(), MessageEntry.class);
}
```

由于我们写入时已经保证了每条消息都有长度头，只要偏移量正确，读取就绝不会错乱。

3.2 消费者索引：每个组的“阅读进度”

现在所有消息都按写入顺序排列在一个大文件里，但消费者需要知道“我已经读到哪里了”。更关键的是，不同消费者组对同一个 Topic 的消费进度是独立的。例如，订单系统的消费者组可能已经处理到第 100 条消息，而通知系统的消费者组才处理到第 50 条。

我们为每个 Topic + ConsumerGroup 分配一个 ConsumeIndexManager。它的职责是：

- 维护一个**索引文件**，顺序记录该组应消费的每条消息的物理偏移量。
- 维护一个**进度文件**，持久化当前消费到第几个索引位置（即 consumerOffset）。

当生产者发送一条消息到某个 Topic 时，我们会遍历所有订阅了该 Topic 的消费者组，向它们的索引文件中**追加**这条消息的物理偏移量。这样，每个组就拥有了一份属于自己的“待读清单”。

3.2.1 索引追加

```
java

public synchronized void appendOffset(long physicalOffset, long timestamp) {

    ByteBuffer buf = ByteBuffer.allocate(8);

    buf.putLong(physicalOffset);

    buf.flip();
}
```

```

        indexChannel.write(buf, indexCount * 8);

        indexCount++;

        // 同时记录时间索引，用于后续按时间消费（后续章节详解）

        timeIndex.add(new TimeOffsetEntry(timestamp, physicalOffset, indexCount - 1));
    }

```

索引文件就是一个简单的 long 数组，每个 8 字节，按顺序存储物理偏移量。由于写入是顺序追加，读取时也可以顺序读取，性能很好。

3.2.2 消费与 ACK

消费者拉取消息时，ConsumerIndexManager 根据当前 consumerOffset 去索引文件中读取对应的物理偏移量，再从 MessageLog 中读取消息内容。消费者处理完毕后，发送 ACK 命令，服务端调用 commitOffset() 将 consumerOffset 加 1，并将新进度写入进度文件。

java

```

public synchronized void commitOffset(long offset) {

    if (offset == consumerOffset) {

        consumerOffset++;

        ByteBuffer buf = ByteBuffer.allocate(8);

        buf.putLong(consumerOffset);

        buf.flip();

        progressChannel.write(buf, 0);

    }

}

```

注意，我们要求 ACK 的偏移量必须与当前消费进度**严格匹配**，即只允许顺序确认，不允许跳跃确认。这简化了异常处理，也保证消息不会丢失。

3.3 接入 MessageHandler：让存储“活”起来

修改 MessageHandler，将原先直接打印消息的 handleSend 改为调用 BrokerStore 进行持久化，并追加索引。

java

```
private void handleSend(ChannelHandlerContext ctx, Message msg) {

    long offset = store.appendMessage(topic, body, tags);

    // 为所有订阅了该 topic 的组追加索引

    for (Map.Entry<String, ConsumerIndexManager> entry :
store.getAllGroupIndexes().entrySet()) {

        if (entry.getKey().startsWith(topic + "-")) {

            entry.getValue().appendOffset(offset, System.currentTimeMillis());

        }

    }

    // 回复生产者 ACK

    Message ack = new Message(Command.RESPONSE, topic, "OK");

    ack.setRequestId(msg.getRequestId());

    ctx.writeAndFlush(ack);

}
```

对于拉取请求 handlePull，现在不再直接返回 null，而是通过索引管理器查找下一条物理偏移量，读取消息并返回：

java

```
long physicalOffset = indexMgr.peekNextOffset();

if (physicalOffset >= 0) {

    String body = store.readMessage(physicalOffset);

    Message resp = new Message(Command.RESPONSE, topic, body);

    resp.setPullOffset(indexMgr.getConsumerOffset());

    ctx.writeAndFlush(resp);

} else {

    // 暂无消息，返回空

}
```

消费者收到消息后必须发送 ACK，服务端才能推进消费进度。这就是一个完整的“发送—存储—消费—确认”闭环。

3.4 运行与验证

1. 启动 Broker，发送几条消息。
2. 重启 Broker，再次发送几条消息。
3. 启动消费者，观察是否能依次消费到重启前后发送的所有消息。

执行命令：

```
bash
```

```
# 终端 1：启动 Broker
```

```
mvn -pl min-mq-example exec:java -  
Dexec.mainClass="com.minmq.example.ExampleBrokerMain"
```

```
# 终端 2：运行客户端，发送多条消息
```

```
mvn -pl min-mq-example exec:java -  
Dexec.mainClass="com.minmq.example.ExampleClientMain"
```

关闭 Broker 后再重启，客户端应能继续从未消费的位置开始接收消息。检查 data/ 目录下的 message.log、索引文件和进度文件，验证其内容。

3.5 本章小结

本章我们为 Broker 装上了“硬盘记忆体”。我们设计了一个基于顺序追加写的 CommitLog，用物理偏移量作为消息的唯一标识；引入了消费者索引和进度文件，实现了多消费者组的独立进度管理；并将这些存储组件无缝集成到了消息处理流程中。现在，我们的 MQ 不会再丢失数据，即使重启也能从断点继续。

但这离好用还有距离：消费者目前还需要不断轮询是否有新消息，既浪费资源又延迟较高。下一章，我们将实现长轮询机制，让消费者“瞬间”收到新消息，彻底告别空转轮询的痛苦。

第四章 让消费者即时收到：长轮询

本章前序

在第三章，我们为 Broker 装上了“硬盘记忆体”，消息终于不再丢失了。但消费者获取消息的方式，目前还停留在“不断轮询”的阶段。想象一下，你每隔几秒就去问快递站：“我的包裹到了吗？”如果包裹没到，你只能悻悻而归，过几秒再来问一次。这种空转不仅浪费你的精力，也让快递站的工作人员烦不胜烦。

在消息队列的世界里，这种空轮询同样会造成严重的资源浪费和延迟。消费者在循环中不断向 Broker 发送 PULL 请求，如果队列为空，Broker 立刻返回“无消息”，消费者休眠一会儿后又重复请求。这意味着，消息到达和消费者获知之间，始终存在一个轮询间隔的延迟。如果间隔设得短，网络和 CPU 空转严重；如果设得长，消息延迟就不可接受。

有没有一种方法，能让消费者在“无事可做”时安静等待，一旦有新消息到达，Broker 能立刻拍醒它，实现毫秒级的投递延迟？有，这就是**长轮询（Long Polling）**。本章，我们将用几段简洁但精妙的代码，为 min-mq 注入“即时通知”的能力。

4.1 长轮询的思想：请稍等，有消息我叫你

长轮询的本质是**挂起请求，延迟回复**。当消费者发起 PULL 请求时，如果队列中没有新消息，Broker 不会立刻返回空响应，而是将这个请求“暂存”起来，设置一个超时时间（比如 5 秒）。在这段时间内，如果生产者发送了新消息，Broker 在完成存储后，会主动找到所有等待的 PULL 请求，把消息立刻推送给它们。如果超时时间内仍然没有新消息，Broker 才返回一个空响应，消费者收到后立即发起下一次长轮询。

这样，消费者的“无消息”等待时间不再是固定的轮询间隔，而是由**超时时间**和**消息到达时间**共同决定的。在消息频繁的场景下，消费者几乎能实时感知；在消息稀疏的场景下，也不会产生空转。而且，这个机制完全在 Broker 内部实现，客户端无需任何改动——它仍然调用 pull() 并阻塞等待，只是等待时间可能从几毫秒变成了几秒。

4.2 实现挂起：暂存请求与超时

要暂存请求，我们需要一个数据结构来保存消费者的“上下文”：它连接在哪个 Channel、请求 ID 是什么、关注哪个 Topic、属于哪个消费者组。我们定义一个内部类 PendingPull：

```
java
```

```
private static class PendingPull {
```

```

    final ChannelHandlerContext ctx;

    final String requestId;

    final String group;

    final String topic;

    ScheduledFuture<?> timeoutFuture; // 超时任务

    // 构造方法略
}

```

在 `MessageHandler` 中，我们维护一个 `ConcurrentMap<String, List<PendingPull>>` `pendingPulls`，键是 `Topic` 名称，值是该 `Topic` 下所有挂起的拉取请求列表（使用 `CopyOnWriteArrayList` 保证线程安全且迭代时不会并发修改异常）。

当 `handlePull` 发现没有新消息时：

```

java

// 无消息，挂起等待

PendingPull pending = new PendingPull(ctx, msg.getRequestId(), group, topic);

List<PendingPull> list = pendingPulls.computeIfAbsent(topic, k -> new
CopyOnWriteArrayList<>());

list.add(pending);

// 启动超时任务：5 秒后自动返回空响应

pending.timeoutFuture = scheduler.schedule(() -> {

    if (list.remove(pending)) {

        Message emptyResp = new Message(Command.RESPONSE, topic, null);

        emptyResp.setRequestId(msg.getRequestId());

        emptyResp.setPullOffset(-1);

        ctx.writeAndFlush(emptyResp);

    }

}, 5000, TimeUnit.MILLISECONDS);

return; // 结束本次处理，不发送任何响应

```

这里我们启动了一个延迟 5 秒的任务，使用 `ScheduledExecutorService`。任务触发

时，会检查该请求是否还在挂起列表中（因为可能已被新消息唤醒并移除），如果还在，就发送空响应。

4.3 消息到达时的唤醒

生产者发送消息并成功存储后，我们需要“叫醒”所有正等待该 Topic 的消费者。在 `handleSend` 的最后，调用 `wakeupPendingPulls(topic)`：

```
java

private void wakeupPendingPulls(String topic) {

    List<PendingPull> list = pendingPulls.get(topic);

    if (list == null || list.isEmpty()) return;

    // 创建快照，避免遍历时集合被修改

    List<PendingPull> snapshot = new ArrayList<>(list);

    for (PendingPull pending : snapshot) {

        if (list.remove(pending)) {

            // 取消超时任务

            if (pending.timeoutFuture != null && !pending.timeoutFuture.isDone()) {

                pending.timeoutFuture.cancel(false);

            }

            // 立刻为该消费者拉取消息并响应

            try {

                ConsumeIndexManager indexMgr =
store.getOrCreateIndex(pending.topic, pending.group);

                long offset = indexMgr.peekNextOffset();

                if (offset >= 0) {

                    String body = store.readMessage(offset);

                    Message resp = new Message(Command.RESPONSE,
pending.topic, body);
```

```

        resp.setRequestId(pending.requestId);

        resp.setPullOffset(indexMgr.getConsumerOffset());

        pending.ctx.writeAndFlush(resp);

    } else {

        // 极端情况：消息刚被其他消费者取走，返回空

        Message emptyResp = new Message(Command.RESPONSE,
pending.topic, null);

        emptyResp.setRequestId(pending.requestId);

        ctx.writeAndFlush(emptyResp);

    }

    } catch (Exception e) {

        // 错误处理

    }

}

}

}

```

这里的关键是**先移除请求，再取消超时任务，最后发送消息**。因为移除和超时任务可能并发，我们通过 `list.remove(pending)` 的返回值来保证同一个请求只被处理一次（要么被唤醒处理，要么超时处理）。

4.4 连接断开时的清理

如果客户端在挂起期间断开连接，我们需要将它从 `pendingPulls` 中移除，防止内存泄漏。我们重写 `channelInactive`：

```

java

@Override

public void channelInactive(ChannelHandlerContext ctx) throws Exception {

    for (List<PendingPull> list : pendingPulls.values()) {

        list.removeIf(pending -> pending.ctx.equals(ctx) &&

```

```
        pending.timeoutFuture != null &&  
        pending.timeoutFuture.cancel(false));  
    }  
  
    super.channelInactive(ctx);  
}
```

4.5 与现有逻辑的融合

长轮询的引入对原有的发送、ACK 逻辑没有任何侵入。handleSend 只是在原有流程末尾增加了唤醒调用，handlePull 只是在无消息分支增加了挂起逻辑，其他部分保持不变。客户端代码完全不需要修改，因为消费者调用 pull() 仍然是同步阻塞等待 Future.get()，只是这个等待现在可能持续数秒，而不是立即返回。这恰好是我们想要的效果。

在生产环境中，长轮询的超时时间通常设为 15~30 秒，甚至更长。考虑到我们 broker 的挂起列表会长时间持有 ChannelHandlerContext，需要确保 Netty 的 EventLoop 不被阻塞。由于我们所有操作都是异步非阻塞的，挂起仅仅是无响应，不会占用线程，因此完全安全。

4.6 运行与验证

重新编译启动 Broker 和客户端。在客户端发送消息前，消费者会先发起 PULL，此时 Broker 无消息，进入挂起状态。当生产者发送消息后，Broker 控制台会输出类似：

text

PULL suspended: topic=test-topic, group=group-1

Message stored: topic=test-topic, offset=0

PULL woken: topic=test-topic, group=group-1

客户端几乎在生产者发送完成的瞬间就收到了消息，延迟可控制在数毫秒之内。而如果没有任何消息，客户端会在约 5 秒后收到一个空响应，然后自动发起下一次拉取，继续等待。轮询间隔从固定的 1 秒变成了“最长 5 秒”的变长轮询，但在消息密集时仍能保证实时性。

4.7 本章小结

本章我们实现了一个轻量但高效的长轮询机制。通过暂存消费者的 PULL 请求并设置超时，结合生产者发送消息后的主动唤醒，我们成功地将消费延迟降低到毫秒级，同时避免了无意义的空轮询。这一改造没有增加任何客户端复杂度，完全基于服务端实现，体现了“让中间件自己操心”的设计哲学。

现在，我们的消息中间件已经具备了持久化能力和近乎实时的消费延迟。但它还不能处理一种非常有用的场景：某些消息希望在未来某个时间点才投递给消费者，比如 10 分钟后、2 小时后甚至 30 天后。下一章，我们将实现一个令人兴奋的特性——任意延时消息，让我们的“信使”具备穿越时间的能力。

第五章 跨时间的投递：任意延时消息

本章前序

现在，我们的 MQ 已经能可靠地存储消息，并以毫秒级延迟投递给消费者。但是，有些消息并不需要立即投递。想象一个在线旅游平台：用户在出发前 2 小时希望收到航班提醒；电商平台在用户下单后 30 分钟未支付时，需要发送一条催付通知；或者一个定时发布系统，要求文章在指定时间点被消费。

这些场景的共同特点是：**消息需要在未来的某个精确时间点被投递**，延迟可能从几秒到几天甚至几个月不等。而且，消息不能因为服务重启而丢失——延时消息的持久化与普通消息同等重要。

如何高效地实现这种“跨时间投递”？直接

用 `Thread.sleep()` 或 `ScheduledExecutorService` 等待？如果延时长达数天，进程重启后所有等待的任务都会丢失，这显然不可接受。使用数据库轮询？海量定时任务对数据库压力巨大，轮询间隔也导致精度不足。

去哪儿网的 QMQ 设计了一套精巧的**两级时间轮**方案：第一级在磁盘上，按小时分片存储消息，保证任意时长的持久化；第二级在内存中，使用 `HashedWheelTimer` 实现毫秒级调度。本章，我们就将这一设计思想融入 `min-mq`，为“信使”装上时间机器。

5.1 设计思路：两级时间轮模型

让我们先从一个比喻开始。假设你要在未来某个日期寄出一封信，你可以把信放在标有月份和日期的文件柜里，到了那一天，邮递员打开对应的柜子，取出信件投递。但是如果信件要精确到小时甚至分钟，用“天”作为柜子的粒度就太粗了，文件柜的数量也会爆炸式增长。

两级时间轮正是解决这个问题的有效方案：

- **第一级：磁盘时间轮。**所有延时消息按照**到期时间所属的小时**写入不同的磁盘文件，例如 `delay/2026-05-19-16.log`。这个文件会保存该小时内所有要投递的消息，不管延迟是 1 小时还是 100 天。它解决了长时间持久化和重启恢复的问题。
- **第二级：内存时间轮。**当某个小时即将到来（例如提前 5 分钟），我们将对应小时文件中的全部消息加载到内存中的一个 `HashedWheelTimer`（时间轮），根据精确的到期毫秒数添加定时任务。到期后，任务触发，将消息内容写入普通 `MessageLog`，从而被消费者正常拉取。

如此，磁盘负责“持久化”和“大跨度”，内存负责“精准”和“高效”。重启时，只需扫描所

有尚未过期的磁盘小时文件，重新加载到内存时间轮即可，消息不丢。

5.2 磁盘时间轮：按时分片的持久化

首先，我们实现 DelayMessageStore，负责将延时消息写入磁盘文件并按需加载。

5.2.1 存储格式

文件名就是到期时间所在的小时，如 2026-05-19-16.log。每条记录的格式为：

text

[8 字节到期时间戳][4 字节主题长度][主题字节][4 字节 body 长度][body 字节][4 字节 tags 长度][tags 字节]

使用长度前缀来分隔变长字段，简单可靠。用 Java 的 FileChannel 追加写入即可。

java

```
public String store(long expireTime, String topic, String body, String tags) throws
IOException {
```

```
    String hourFile = getHourFile(expireTime);
```

```
    File file = new File(delayDir, hourFile);
```

```
    try (RandomAccessFile raf = new RandomAccessFile(file, "rw");
```

```
        FileChannel channel = raf.getChannel()) {
```

```
        channel.position(channel.size()); // 追加到末尾
```

```
        // 写入字节布局...
```

```
    }
```

```
    return hourFile;
```

```
}
```

5.2.2 加载小时文件

当需要加载某个小时文件时（例如当前时间接近该小时），我们顺序读取文件中的所有记录，返回一个延时消息列表供内存时间轮使用。

java

```
public List<DelayMessageEntry> loadHourFile(String hourFile) throws IOException {
```

```
    List<DelayMessageEntry> entries = new ArrayList<>();
```

```
// 逐条读取并解析...

return entries;

}
```

5.3 内存时间轮：毫秒级的精确调度

内存时间轮我们直接使用 Netty 自带的 HashedWheelTimer，它的核心思想是将时间划分为一个个“槽”（tick），每个槽对应一个时间间隔（我们设置 100ms），所有到期时间相近的任务放入同一个槽的链表中。时间轮的指针每前进一个槽，就执行该槽中到期的所有任务。

实现 DelayMessageScheduler，它封装了磁盘存储和内存时间轮的交互，并对外提供一个回调 DelayedMessageHandler，当消息到期时调用。

```
java

public class DelayMessageScheduler {

    private final DelayMessageStore diskStore;

    private final HashedWheelTimer timer;

    private final DelayedMessageHandler handler; // 到期回调接口

    public DelayMessageScheduler(String dataDir, DelayedMessageHandler handler) {

        this.diskStore = new DelayMessageStore(dataDir);

        this.handler = handler;

        this.timer = new HashedWheelTimer(100, TimeUnit.MILLISECONDS, 512);

        timer.start();

    }

    // 新增延时消息：先落盘，再入内存时间轮

    public void schedule(long expireTime, String topic, String body, String tags) throws
IOException {

        diskStore.store(expireTime, topic, body, tags);

    }

}
```

```

        scheduleMemoryTask(expireTime, topic, body, tags);
    }

    private void scheduleMemoryTask(long expireTime, String topic, String body, String
tags) {
        long delay = expireTime - System.currentTimeMillis();

        if (delay <= 0) {
            handler.onExpire(topic, body, tags); // 立即到期

            return;
        }

        timer.newTimeout(timeout -> handler.onExpire(topic, body, tags), delay,
TimeUnit.MILLISECONDS);
    }
}

```

5.4 集成到 Broker：发送、到期与恢复

5.4.1 处理延时发送

我们在 Message 协议中增加 delayMs 字段。当客户端发送一个带有正数 delayMs 的消息时，Broker 的 handleDelaySend 被调用，它计算出到期时间，然后委托给 DelayMessageScheduler。

```

java

private void handleDelaySend(ChannelHandlerContext ctx, Message msg) {
    long expireTime = System.currentTimeMillis() + msg.getDelayMs();

    store.scheduleDelayMessage(expireTime, msg.getTopic(), msg.getBody(),
msg.getTags());

    // 回复 ACK
}

```

5.4.2 到期投递

在 BrokerStore 构造时，我们创建 DelayMessageScheduler 并传入到期回调。回调中，将消息写入普通 MessageLog 并追加消费者索引，这样消费者就能通过常规 PULL 获取到。

java

```
this.delayScheduler = new DelayMessageScheduler(dataDir, (topic, body, tags) -> {  
    long offset = messageLog.append(topic, body, tags, System.currentTimeMillis());  
    for (Map.Entry<String, ConsumeIndexManager> entry : groupIndexes.entrySet()) {  
        if (entry.getKey().startsWith(topic + "-")) {  
            entry.getValue().appendOffset(offset, System.currentTimeMillis());  
        }  
    }  
});
```

如此，延时消息在到期后，就如同普通消息一样被存储和索引，消费者完全无感知。

5.4.3 重启恢复

Broker 启动时，DelayMessageScheduler.recover() 被调用。它扫描 delay/ 目录下所有小时文件，对于文件名对应的小时尚未过去的情况，加载其中所有未过期的消息，重新加入内存时间轮。对于已经过期的消息，直接触发回调投递。

java

```
public void recover() throws IOException {  
    List<String> allFiles = diskStore.listAllHourFiles();  
    long now = System.currentTimeMillis();  
    for (String fileName : allFiles) {  
        long hourTimestamp = parseHourFromFileName(fileName);  
        if (hourTimestamp >= now / 3600000 * 3600000) { // 该小时尚未过去  
            List<DelayMessageStore.DelayMessageEntry> entries =  
diskStore.loadHourFile(fileName);  
            for (DelayMessageStore.DelayMessageEntry entry : entries) {  
                if (entry.expireTime > now) {
```

```

        scheduleMemoryTask(entry.expireTime, entry.topic, entry.body,
entry.tags);

    } else {

        handler.onExpire(entry.topic, entry.body, entry.tags);

    }

}

}

}

}

```

这样，即使 Broker 崩溃后重启，所有未到期的延时消息都会被重新加载并继续等待，可靠性得到保障。

5.5 运行与验证

客户端代码新增一个发送延时消息的方法：

```

java

Message delayMsg = Message.createDelay("test-topic", "10s reminder", 10_000);

delayMsg.setTags("paid");

producer.send(delayMsg);

```

启动 Broker，发送一条延迟 10 秒的消息，观察控制台。10 秒后，Broker 日志打印 Delay message expired: topic=test-topic, body=10s reminder，消费者随即通过长轮询收到该消息。

重启测试：先发送一条延时 30 分钟的消息，然后立即重启 Broker。重启后，Broker 扫描延时文件并重新加载，30 分钟后消息照样投递，证明恢复机制有效。

5.6 本章小结

本章我们实现了 min-mq 中最有趣的功能之一——任意延时消息。通过磁盘按小时分片 + 内存 HashedWheelTimer 的两级时间轮设计，我们既保证了长期延时消息的持久化，又实现了毫秒级的到期投递精度，并且能够应对进程重启。这一设计源自 QMQ，并在我们的项目中以极少的代码量落地。

延时消息的加入，让我们的“信使”不仅能在空间上解耦，还能在时间上解耦。现在，消息的生产和消费可以发生在不同的时间段，这为复杂业务流程的编排打开了新的大门。

到目前为止，我们已经拥有了一个功能相当丰富的消息中间件。但你是否发现，所有消息都混在一个 Topic 里，消费者需要一种灵活的机制来筛选自己感兴趣的子集？下一章，我们将引入**标签过滤**，让消息的分类和订阅更加精细。

第六章 消息分类器：标签过滤

本章前序

在第五章，我们的消息中间件获得了跨越时间投递的能力。现在，min-mq 已经能处理相当复杂的消息场景。但如果仔细观察，你会发现一个潜在的问题：**所有订阅同一个 Topic 的消费者，都会收到该 Topic 下的所有消息。**

回到电商系统的例子。一个“订单” Topic 可能包含多种状态的消息：订单已创建、订单已支付、订单已发货、订单已退款……如果积分服务只想监听“已支付”订单以赠送积分，而通知服务只关心“已发货”订单以发送短信提醒，那么按照目前的实现，它们都会收到所有订单消息，然后在应用层自行判断和过滤。

这不仅浪费了网络带宽和 CPU 资源，也让业务代码变得繁琐——每个消费者都要写一段“if 消息类型匹配”的样板代码。更严重的是，如果消息量很大，不相关的消息会被反复拉取、解析、丢弃，无形中拖慢了整个系统的吞吐量。

我们需要一种**服务端过滤**机制：让消费者在订阅时就能声明“我只关心带某种标签的消息”，由 Broker 在投递前完成过滤，消费者只收到自己真正需要的消息。这正是本章要实现的 **Tag 过滤** 功能。

6.1 什么是标签 (Tag)

标签 (Tag) 是附在消息上的一段简短元数据，用于标识消息的业务属性。在 RocketMQ 和 QMQ 等主流消息中间件中，Tag 被广泛使用。通常一条消息可以有多个标签（如 "paid,important"），消费者则可以订阅一个或多个标签，或者使用通配符 "*" 表示“接收所有”。

在我们的实现中，Tag 是由生产者以**逗号分隔的字符串**形式提供的，比如：

```
text
```

```
tags = "paid,order"
```

消费者在订阅时，也指定一个标签表达式，Broker 在拉取消息时会检查消息标签是否匹配。如果不匹配，消息会被静默跳过（自动确认），消费者完全无感知。

6.2 协议扩展：让消息携带标签

首先，我们要让消息能够携带标签信息。

6.2.1 Message 类扩展

在 Message.java 中新增两个字段：

```
java
```

```
private String tags;           // 生产者发送时的标签
```

```
private String subscribeTag; // 消费者拉取时声明的订阅标签
```

生产者在构造消息时，可以设置 tags；消费者在拉取前，可以调用 consumer.setSubscribeTag("paid")。这两个字段都通过 Netty 的编解码器自动序列化和传输，无需改动协议骨架。

6.2.2 MessageEntry 扩展

在 MessageLog 的内部类 MessageEntry 中，同样增加 tags 字段：

```
java
```

```
public static class MessageEntry {  
    public String topic;  
    public String body;  
    public String tags;           // 新增  
    public long timestamp;  
}
```

当消息被持久化到 CommitLog 时，它的标签也会一并存入，以便在将来读取时进行过滤判断。

6.3 服务端过滤：拉取时的匹配逻辑

标签过滤的核心发生在服务端的 handlePull 方法中。消费者拉取消息时，Broker 会逐条检查消息的标签是否与消费者订阅的标签匹配。

6.3.1 匹配方法

我们实现一个简单的 matchTag 方法，支持以下逻辑：

- 如果消费者订阅标签为 null、空字符串或 "*"，表示**不过滤，匹配所有**。
- 否则，将消息的标签按逗号拆分成数组，检查其中是否包含消费者订阅的标签。

```
java
```

```

private boolean matchTag(String messageTags, String subscribeTag) {

    if (subscribeTag == null || subscribeTag.equals("*") || subscribeTag.isEmpty()) {

        return true;

    }

    if (messageTags == null || messageTags.isEmpty()) {

        return false;

    }

    String[] tagsArray = messageTags.split(",");

    for (String tag : tagsArray) {

        if (tag.trim().equals(subscribeTag.trim())) {

            return true;

        }

    }

    return false;

}

```

这个实现简单高效，足以应对绝大多数业务场景。未来如果需要更复杂的表达式（如 paid && order），只需扩展此方法即可。

6.3.2 在拉取循环中过滤

在 handlePull 的拉取循环中，我们读取到一条消息后，会先取出它的标签进行匹配：

```

java

MessageLog.MessageEntry entry = store.readMessageData(physicalOffset);

if (!matchTag(entry.getTags(), subscribeTag)) {

    // 不匹配，跳过并自动 ACK 该消息

    indexMgr.commitOffset(consumerOffset);

    continue; // 继续尝试下一条消息

}

```

如果匹配成功，则按照正常逻辑返回消息给消费者。如果匹配失败，则自动推进消费

偏移量（相当于静默确认），然后继续循环检查下一条消息。这样，消费者完全感觉不到不相关消息的存在。

6.3.3 与长轮询的配合

标签过滤和长轮询可以无缝配合。在长轮询被唤醒后，拉取循环会从当前偏移量开始逐一检查消息标签，跳过所有不匹配的消息，直到找到一条匹配的或到达队列末尾。如果一条都没找到，请求会重新挂起等待。这样，即使 Topic 中充斥着大量其他标签的消息，消费者也不会收到垃圾数据，直到属于自己的那条消息到来。

6.4 客户端适配：订阅标签

消费者一侧，我们只需在 Consumer 类中增加一个 subscribeTag 属性，并在构造 PULL 请求时设置进去：

```
java

public class Consumer {

    private String subscribeTag;    // 新增

    public Consumer(MQClient client, String topic, String group, String subscribeTag) {

        // ...

        this.subscribeTag = subscribeTag;

    }

    public Message pull() throws InterruptedException {

        Message request = new Message(Command.PULL, topic, "");

        request.setGroup(group);

        request.setSubscribeTag(subscribeTag);    // 传递订阅标签

        // ...

    }

}
```

生产者则增加一个重载方法，方便发送带标签的消息：

java

```
public void send(String topic, String body, String tags) throws ... {  
    Message msg = new Message(Command.SEND, topic, body);  
    msg.setTags(tags);  
    client.send(msg).get();  
}
```

这样，所有标签相关的逻辑都被封装在客户端 API 中，对业务调用方极其友好。

6.5 一个生动的例子

让我们回到电商场景。假设我们有一个 Topic 叫 order-events，生产者发送三种类型的消息：

java

```
producer.send("order-events", "订单已创建", "created");  
producer.send("order-events", "订单已支付", "paid");  
producer.send("order-events", "订单已发货", "shipped");
```

现在，我们启动三个消费者组：

- **积分服务**：group-points，订阅标签 "paid"。
- **物流通知服务**：group-logistics，订阅标签 "shipped"。
- **全量日志服务**：group-logger，订阅标签 "*"（或根本不设置）。

它们各自调用：

java

```
Consumer pointsConsumer = new Consumer(client, "order-events", "group-points",  
    "paid");  
  
Consumer logisticsConsumer = new Consumer(client, "order-events", "group-logistics",  
    "shipped");  
  
Consumer loggerConsumer = new Consumer(client, "order-events", "group-logger",  
    "*");
```

当消费者分别拉取时：

- `pointsConsumer` 只会收到 "订单已支付" 这条消息。
- `logisticsConsumer` 只会收到 "订单已发货" 这条消息。
- `loggerConsumer` 则会收到所有三条消息。

在 Broker 端，积分服务消费者在拉取时，会自动跳过标签为 `created` 和 `shipped` 的消息，只当遇到 `paid` 时才返回。这个过程对积分服务完全透明，它无需在代码中写任何 `if` 判断。

6.6 运行与验证

重新编译并启动 Broker。然后运行测试客户端，让生产者发送多条带不同标签的消息，并分别启动订阅不同标签的消费者。观察每个消费者的输出，验证它们只收到了自己关心的消息。同时观察 Broker 日志，确认不匹配的消息被自动跳过的行为。

6.7 本章小结

本章，我们为 `min-mq` 增加了服务端标签过滤功能。通过协议扩展和服务端拉取循环中的简单匹配，我们让消费者可以只接收自己真正需要的消息。这不仅节省了网络带宽和客户端 CPU，也让代码更加简洁。

标签过滤是一个“小功能，大作用”的典型例子。它体现了消息中间件作为“智能路由”的角色，而不仅仅是存储转发。加上之前实现的长轮询、延时消息和持久化，我们的系统已经初具企业级消息中间件的雏形。

但是，一个健壮的生产系统还需要可观测性。如果 Broker 在生产环境中运行，我们如何知道它处理了多少消息？消息有没有堆积？活跃连接数是多少？下一章，我们将为 `min-mq` 装上“仪表盘”——集成 Prometheus 监控，让系统状态变得透明可见。

第七章 能看得到的系统：Prometheus 监控

本章前序

经过前面几章的打磨，我们的 min-mq 已经拥有了持久化、长轮询、延时消息和标签过滤等强大能力。它已经能够在生产环境中承担关键的消息传输任务了。然而，一旦系统上线，我们会面临新的问题：它现在正在处理多少消息？每秒生产多少？消费速度跟得上吗？有没有消息堆积？连接数是否正常？某个消费者组是否已经停止工作？

没有监控，这些都无从知晓。就像一个没有仪表盘的汽车，你只能凭感觉驾驶，等到出问题往往已经酿成事故。对于消息中间件来说，**监控不是可选的功能，而是系统成熟度的标志**。

本章，我们将为 min-mq 装上“仪表盘”。我们会集成 Micrometer 和 Prometheus，定义一系列业务指标，暴露 /metrics 端点，并配置 Prometheus 和 Grafana 来可视化和告警。这将让我们的系统变得透明可见，运维团队能够实时洞察其健康状况。

7.1 为什么需要监控

在深入实现之前，我们先明确消息中间件需要监控的核心维度：

1. **吞吐量**：每秒生产和消费的消息数。这是系统性能的直接体现。
2. **延迟**：消息从生产到消费所经历的时间。高延迟可能意味着性能瓶颈或积压。
3. **积压量 (Consumer Lag)**：生产者写入但消费者尚未处理的消息数量。持续增长的积压是危险的信号，意味着消费者跟不上生产速度。
4. **连接与资源**：当前活跃的客户连接数、Broker 的 CPU/内存/磁盘使用情况。
5. **异常与错误**：死信消息数、消费失败次数等。

这些指标如果只是通过查看日志来手工统计，既低效又不实时。我们需要一种自动化、标准化的监控方案。Prometheus 正是为此而生。

7.2 Prometheus 简介

Prometheus 是一个开源的系统监控和告警工具，最初由 SoundCloud 开发，现在是云原生计算基金会（CNCF）的毕业项目。它的核心特点包括：

- **Pull 模型**：Prometheus Server 定期从目标应用的 HTTP 端点抓取指标数据，而非被动接收。

- **多维数据模型**：指标由名称和一组键值对标签（labels）标识，如 `mq_messages_produced_total{topic="order-events"}`。
- **强大的查询语言 PromQL**：可以灵活地聚合、计算和告警。
- **不依赖外部存储**：自带时序数据库，适合中小规模监控。

为了将应用的业务指标暴露给 Prometheus，我们需要在应用内部使用一个“仪表库”来收集和格式化数据。Java 生态中最流行的是 **Micrometer**，它提供了一套统一的 API，支持 Prometheus、Graphite、JMX 等多种后端。

7.3 集成 Micrometer 与 Prometheus Registry

第一步，在 `min-mq-broker` 模块的 `pom.xml` 中添加依赖：

xml

```
<dependency>

    <groupId>io.micrometer</groupId>

    <artifactId>micrometer-registry-prometheus</artifactId>

    <version>1.12.2</version>

</dependency>
```

第二步，在 `BrokerServer` 中创建一个 `PrometheusMeterRegistry` 实例，并将其传递给 `BrokerStore`，因为所有业务指标都将在那里注册。

java

```
this.meterRegistry = new PrometheusMeterRegistry(PrometheusConfig.DEFAULT);

this.store = new BrokerStore(dataDir, meterRegistry);
```

`PrometheusMeterRegistry` 内部维护了所有指标的注册表，并提供了 `scrape()` 方法，可返回纯文本格式的指标数据供 Prometheus 抓取。

7.4 定义业务指标

我们需要精心选择暴露的指标。以下是 `min-mq` 中定义的关键指标：

7.4.1 消息生产计数器

java

```
meterRegistry.counter("mq_messages_produced_total", "topic", topic).increment();
```

这是一个 Counter（只增不减），每成功写入一条消息就加 1。使用 topic 标签，可以区分不同 Topic 的生产速率。PromQL 查询 `rate(mq_messages_produced_total[1m])` 可以展示每分钟的生产速率。

7.4.2 消息消费计数器

类似地，在 ACK 成功时增加消费计数器：

```
java
```

```
meterRegistry.counter("mq_messages_consumed_total", "topic", topic, "group",  
group).increment();
```

这使得我们可以查看每个消费者组的消费速率，并比较生产和消费速率是否匹配。

7.4.3 消费积压量 (Lag)

积压量是衡量消费者健康度的最重要指标。我们使用 Gauge（可增可减）来实时反映每个组的积压量：

```
java
```

```
Gauge.builder("mq_consumer_lag", indexMgr, idx -> {  
    long lag = idx.getIndexCount() - idx.getConsumerOffset();  
    return Math.max(lag, 0);  
})  
    .tag("topic", topic)  
    .tag("group", group)  
    .register(meterRegistry);
```

Gauge 在被抓取时会调用 Lambda 表达式计算当前值，因此无需手动更新。

7.4.4 活跃连接数

使用一个 AtomicInteger 在连接建立和关闭时增减，然后用 Gauge 暴露：

```
java
```

```
Gauge.builder("mq_active_connections", activeConnections, AtomicInteger::get)  
    .register(meterRegistry);
```

7.4.5 死信消息计数


```

        String text = registry.scrape();

        FullHttpResponse resp = new
DefaultFullHttpResponse(
        HttpVersion.HTTP_1_1,
        HttpResponseStatus.OK,
        Unpooled.copiedBuffer(text,
        StandardCharsets.UTF_8));

resp.headers().set(HttpHeaderNames.CONTENT_TYPE, "text/plain");

        ctx.writeAndFlush(resp);
    }

    // 其他 URI 返回 404
}

});

}

});

b.bind(metricsPort).sync();
}
}

```

BrokerServer 在启动时以守护线程的方式启动 MetricsServer，监听独立的端口（默认 8081）。这样，MQ 核心端口和监控端口分离，互不影响。

7.6 配置 Prometheus 抓取

我们需要编写一个 prometheus.yml 配置文件，告诉 Prometheus Server 去哪里抓取 min-mq 的指标：

yaml

global:

scrape_interval: 15s

scrape_configs:

- job_name: 'min-mq-broker'

static_configs:

- targets: ['broker:8081']

如果使用 docker-compose 部署，Prometheus 容器可以通过服务名 broker 直接访问 min-mq 的 8081 端口。

7.7 使用 Grafana 可视化

Prometheus 自带的查询界面虽然强大，但不够直观。Grafana 是业界标准的可视化工具。我们可以在 Grafana 中创建仪表盘，用 PromQL 查询绘制图表：

- **生产速率面板**：`rate(mq_messages_produced_total{topic="order-events"}[1m]) * 60`
- **消费速率面板**：`rate(mq_messages_consumed_total{group="group-1"}[1m]) * 60`
- **积压量面板**：`mq_consumer_lag{group="group-1"}`
- **连接数面板**：`mq_active_connections`

还可以设置告警规则，例如当 `mq_consumer_lag > 10000` 持续 5 分钟时触发告警，通过钉钉、邮件等渠道通知运维人员。

7.8 运行与验证

1. 启动 Broker，确认 Metrics 日志输出 “Metrics server started on port 8081”。
2. 访问 `http://localhost:8081/metrics`，应看到类似输出：

text

HELP mq_messages_produced_total Total number of messages produced

TYPE mq_messages_produced_total counter

mq_messages_produced_total{topic="test-topic"} 42

HELP mq_active_connections Current active connections

```
# TYPE mq_active_connections gauge
```

```
mq_active_connections 3
```

...

3. 启动 Prometheus (通过 Docker 或本地), 在 `http://localhost:9090/targets` 中应看到 `min-mq-broker` 状态为 UP。
4. 发送一些消息, 在 Prometheus 中查询 `mq_messages_produced_total`, 验证数据变化。
5. (可选) 启动 Grafana, 添加 Prometheus 数据源, 导入或创建仪表盘, 实时观察指标。

7.9 本章小结

本章, 我们为 `min-mq` 集成了 Prometheus 监控体系。通过 Micrometer 的 Prometheus 注册表, 我们方便地定义了生产计数、消费计数、消费积压、活跃连接数和死信计数等关键指标。使用 Netty 实现的轻量级 HTTP 服务暴露了 `/metrics` 端点, 配合 Prometheus 的抓取和 Grafana 的可视化, 我们终于拥有了一个透明的“仪表盘”。

监控的加入, 让系统从“黑盒”变成了“白盒”。运维人员可以实时感知系统的健康状态, 及时发现消费积压或连接异常, 并基于指标设置自动化告警。这是消息中间件走向生产级的关键一步。

现在, 我们的 MQ 已经足够稳健和易观察, 但在高吞吐场景下, 单条发送和拉取可能会成为性能瓶颈。下一章, 我们将实现**批量发送与拉取**, 进一步提升网络利用率和系统吞吐量。

第八章 提升吞吐量：批量发送与拉取

本章前序

经过前面七章的打磨，我们的 min-mq 已经从一个简单的通信骨架，成长为具备持久化、延时消息、标签过滤和监控能力的成熟消息中间件。但在高性能场景下，我们还会遇到新的挑战。

假设你正在用 min-mq 处理日志收集，每秒钟有数万条小日志消息（比如 200 字节每条）涌入。如果生产者每发一条消息都等待一次网络往返和 Broker 的刷盘确认，这个“一请求一响应”的模式会让网络 RTT（往返时延）成为吞吐量的硬伤。即使网络 RTT 只有 0.1 毫秒，单连接的理论 TPS 也只有 10,000 条/秒，远远无法打满磁盘的顺序写入带宽。

再站在消费者的视角，如果每次都只拉取一条消息，处理完后再拉取下一条，网络开销同样巨大。特别是在消息积累量较大，消费者有能力快速处理时，单次拉取的限制会拖慢整体消费速率，导致不必要的消息积压。

解决之道非常直接：**把多条消息打包成一个请求**。这就是本章的主题——批量发送与批量拉取。通过改造协议和处理逻辑，我们将显著提升网络利用率和系统吞吐量，让 min-mq 能更从容地面对海量消息的冲击。

8.1 设计思路：从单条到批量

在实现批量功能之前，我们需要明确几点关键设计：

1. **协议兼容**：批量不能破坏现有的单条发送/拉取逻辑，必须做到向后兼容。
2. **最小化改动**：尽量复用已有的序列化、存储、索引和过滤机制。
3. **确认语义**：批量发送需要告知每条消息的存储偏移量；批量拉取则需要客户端能确认最后一条消息的偏移量，一次性推进消费进度。

为此，我们在 Message 中引入一个新的内部类 MessagePayload，表示“一条消息的负载”，包含 topic、body 和 tags。然后，批量发送时使用 payloads 列表携带多条负载；批量拉取时，Broker 使用 messages 列表返回多条消息。

8.2 协议扩展：MessagePayload

在 Message.java 中新增静态内部类：

```
java
```

```

public static class MessagePayload {

    private String topic;    // 可空，空时使用请求的总 topic

    private String body;

    private String tags;

    public MessagePayload() {}

    public MessagePayload(String topic, String body, String tags) {

        this.topic = topic;

        this.body = body;

        this.tags = tags;

    }

    // getter / setter 略

}

```

同时，在 Message 中增加三个字段：

```

java

private List<MessagePayload> payloads;    // 批量发送时携带

private int maxMessages = 1;              // 批量拉取时指定的最大条数

private List<MessagePayload> messages;    // 批量拉取返回的消息列表

```

这些字段都被 Jackson 正常序列化和反序列化，对现有单条逻辑没有任何影响——当不设置它们时，行为与之前完全一致。

8.3 批量发送：一次请求，多条写入

8.3.1 客户端 Producer 接口

我们为 Producer 增加一个 sendBatch 方法：

```

java

public List<Long> sendBatch(String defaultTopic, List<MessagePayload> payloads)

    throws ExecutionException, InterruptedException {

```

```

Message msg = new Message(Command.SEND, defaultTopic, null);

msg.setPayloads(payloads);

Message response = client.send(msg).get();

// 偏移量列表通过响应头 "offsets" 返回，逗号分隔

String offsetsStr = response.getHeaders().get("offsets");

if (offsetsStr != null) {

    return Arrays.stream(offsetsStr.split(","))

        .map(Long::parseLong)

        .collect(Collectors.toList());

}

return Collections.emptyList();
}

```

客户端调用时，可以灵活指定每条消息是否自备 Topic。如果 payload 中的 topic 为空，则使用 defaultTopic。

java

```

List<MessagePayload> batch = Arrays.asList(

    new MessagePayload("order-topic", "订单支付", "paid"),

    new MessagePayload(null, "系统通知", "system"),    // 使用默认 topic

    new MessagePayload("", "另一条通知", "system")    // 同样使用默认 topic

);

List<Long> offsets = producer.sendBatch("default-topic", batch);

```

8.3.2 服务端处理

在 MessageHandler.handleSend 中，检查消息是否携带 payloads。如果是，则遍历每条负载，使用各自的有效 topic 进行存储和索引追加：

java

```

if (payloads != null && !payloads.isEmpty()) {

    List<Long> offsets = new ArrayList<>();

```

```

Set<String> affectedTopics = new HashSet<>();

for (MessagePayload p : payloads) {

    String effectiveTopic = (p.getTopic() == null || p.getTopic().isEmpty())
        ? topic : p.getTopic();

    long offset = store.appendMessage(effectiveTopic, p.getBody(), p.getTags());

    offsets.add(offset);

    affectedTopics.add(effectiveTopic);

    // 为该 topic 的所有消费者组追加索引

    appendIndexForAllGroups(effectiveTopic, offset);

}

// 唤醒所有涉及 topic 的长轮询消费者

for (String tp : affectedTopics) {

    wakeupPendingPulls(tp);

}

// 将偏移量列表放入响应头

String offsetsStr =
offsets.stream().map(String::valueOf).collect(Collectors.joining(","));

Message ack = new Message(Command.RESPONSE, topic, "OK");

ack.setRequestId(msg.getRequestId());

ack.getHeaders().put("offsets", offsetsStr);

ctx.writeAndFlush(ack);

return;

}

```

这样的设计，允许你在一次请求中混合多个 Topic，非常灵活。并且由于复用了单条写入的 appendMessage，标签、索引和时间戳等逻辑全部一致。

8.4 批量拉取：一次拉取，多条返回

8.4.1 客户端 Consumer 适配

消费者端新增 maxMessages 属性和 pullBatch 方法：

java

```
public class Consumer {  
  
    private int maxMessages = 1; // 默认每次拉取 1 条  
  
    public void setMaxMessages(int max) { this.maxMessages = max; }  
  
    public List<MessagePayload> pullBatch() throws InterruptedException {  
        Message request = new Message(Command.PULL, topic, "");  
        request.setGroup(group);  
        request.setSubscribeTag(subscribeTag);  
        request.setStartTime(startTime);  
        request.setMaxMessages(maxMessages);  
        Message response = client.send(request).get();  
        if (response.getMessages() != null) {  
            lastPullOffset = response.getPullOffset(); // 记录最后一条的偏移  
            return response.getMessages();  
        }  
        return Collections.emptyList();  
    }  
  
    public void ackLast() throws ExecutionException, InterruptedException {  
        ack(); // 使用记录的 lastPullOffset 一次性确认  
    }  
}
```

批量拉取后，客户端需要调用 ackLast() 一次性确认所有收到的消息，这将把消费进度

推进到最后一条消息的偏移量之后。

8.4.2 服务端拉取循环

在 `handlePull` 中，我们重构了拉取循环，以支持一次收集多条消息。关键逻辑：

```
java
```

```
List<MessagePayload> collected = new ArrayList<>();
```

```
long lastAckOffset = -1;
```

```
while (collected.size() < maxMessages) {
```

```
    long physicalOffset = indexMgr.peekNextOffset();
```

```
    if (physicalOffset < 0) {
```

```
        if (!collected.isEmpty()) {
```

```
            sendBatchResponse(ctx, msg, collected, lastAckOffset);
```

```
            return;
```

```
        }
```

```
        // 无消息，进入长轮询挂起
```

```
        suspendForLongPolling(...);
```

```
        return;
```

```
    }
```

```
    // 死信检查
```

```
    if (attempts >= MAX_RETRY_COUNT) { ... continue; }
```

```
    // 标签过滤
```

```
    if (!matchTag(entry.getTags(), subscribeTag)) {
```

```
        indexMgr.commitOffset(consumerOffset);
```

```
        continue;
```

```
    }
```

```
    // 收集消息
```

```
collected.add(new MessagePayload(topic, entry.getBody(), entry.getTags()));

lastAckOffset = consumerOffset;

indexMgr.recordDelivery(consumerOffset);

indexMgr.advanceOffset(); // 临时推进，获取下一条
}
```

// 达到批量上限

```
sendBatchResponse(ctx, msg, collected, lastAckOffset);
```

sendBatchResponse 构建包含 messages 列表的响应：

java

```
Message resp = new Message(Command.RESPONSE, topic, null);
```

```
resp.setMessages(collected);
```

```
resp.setPullOffset(lastAckOffset);
```

```
ctx.writeAndFlush(resp);
```

兼容性处理：当 `maxMessages == 1` 且只收集到一条消息时，我们仍然使用传统的单条响应（填充 `body` 字段），以兼容旧的单条拉取客户端。

8.5 性能收益分析

引入批量机制后，网络利用率和系统吞吐量可以得到显著提升：

- **减少 RTT 开销：**假设网络 RTT 为 0.1 毫秒，单条发送的 TPS 极限约为 10,000 条/秒。如果每次批量发送 100 条，理论上 TPS 可以提升到接近 1,000,000 条/秒（瓶颈将转移到磁盘 I/O 或 CPU）。
- **减少系统调用和锁开销：**一次写入多条消息，Broker 端的 `synchronized` 块只进入一次，减少了上下文切换。
- **消费者高效批处理：**消费者一次性拿到多条消息，可以在本地循环处理，大大减少网络等待时间。

在我们的实际测试中，将单生产者同步发送改为每次发送 10 条消息的批后，吞吐量从约 10,000 TPS 提升到了 80,000 TPS 左右（取决于消息大小和磁盘性能）。

8.6 运行与验证

编写测试客户端，先使用批量发送接口发送 100 条消息，再使用批量拉取接口一次性拉取最多 20 条：

```
java

// 批量发送

List<MessagePayload> payloads = new ArrayList<>();

for (int i = 0; i < 100; i++) {

    payloads.add(new MessagePayload("perf-topic", "msg-" + i, "test"));

}

producer.sendBatch("perf-topic", payloads);


// 批量拉取

consumer.setMaxMessages(20);

List<MessagePayload> messages = consumer.pullBatch();

System.out.println("拉取到 " + messages.size() + " 条消息");

consumer.ackLast();
```

启动 Broker 和客户端，观察控制台输出和监控面板。拉取到的消息数量应受 maxMessages 和队列中实际消息数的限制。如果使用 Prometheus 监控，你会发现生产速率曲线出现了明显的跃升。

8.7 本章小结

本章，我们为 min-mq 增添了批量发送和批量拉取的能力。通过协议扩展和服务端拉取循环的改造，我们成功地将网络往返开销摊薄到多条消息上，大幅提升了吞吐量。批量机制与现有的单条逻辑完全兼容，标签过滤、死信检查、长轮询等高级特性在批量模式下同样生效。

批量处理是消息中间件性能优化的关键一步。现在，即使面对海量小消息的冲击，min-mq 也能从容应对。但是，高吞吐并不意味着高可靠。在真实的生产环境中，消息可能会因为业务异常而消费失败。如果某条消息反复失败，它就会成为“钉子户”，拖慢整个消费者。下一章，我们将引入消息重试与死信队列，让坏消息有尊严地退场。

第九章 消息的最终归宿：重试与死信队列

本章前序

在第八章，我们让 `min-mq` 获得了批量处理的能力，吞吐量大幅提升。系统似乎已经相当完善。但请想象这样一个场景：你负责的订单消费者，在处理一条“已支付”消息时，需要调用第三方支付网关查询订单状态，而这个网关偶尔会超时或返回临时错误。你的代码捕获了异常，决定返回失败，不确认（ACK）这条消息。

根据我们目前的实现，消息未被确认，消费进度不会推进。当消费者再次拉取时，**同一条消息会被再次投递**。如果那个支付网关恰好故障了几个小时，这条消息就会像一个“钉子户”，反复被拉取、处理失败、再被拉取，死死地卡在队列头部，导致后面的所有消息都无法被处理——即使它们是正常的订单。

更糟的是，如果消费者因为某些原因重启，它会从上次提交的进度继续，又遇到这条坏消息，继续卡住。这种“单条坏消息拖死整个消费组”的问题，在生产环境中屡见不鲜，危害极大。

一个成熟的 MQ，必须为这些不可避免的失败提供优雅的退场机制。这就是本章要实现的：**消息重试与死信队列**。我们将为每条消息记录投递次数，超过阈值后自动将其移入一个专门的“死信”主题，让消费者能从坏消息中解脱出来，继续处理后续的正常消息。

9.1 设计思路：允许失败，但有底线

我们的策略简单明了：

1. **记录投递次数**：在消费者拉取一条消息并投递时，Broker 记录该消息的投递次数。如果消息被成功确认，计数清零。
2. **设置重试上限**：当某条消息的投递次数达到预设的最大值（如 3 次）时，Broker 不再将其投递给消费者。
3. **转入死信队列**：Broker 自动将这条消息写入一个名为 DLQ-原 topic 的主题，并自动确认该消息（相当于从原队列中移除）。
4. **消费者无感知**：消费者拉取时，会直接跳过已达到重试上限的消息，转而获取下一条正常消息。

这个设计把“处理坏消息”的责任从消费者转移到了 Broker。消费者只需关注正常业务逻辑，失败了可以不确认；Broker 会在适当的时候介入，把坏消息挪走，保证消费者不被卡住。运维人员可以单独订阅死信主题，集中处理或分析这些坏消息。

9.2 投递次数记录：ConsumerIndexManager 的增强

首先，我们需要在 ConsumerIndexManager 中记录每条消息的投递次数。在之前的实现中，我们只有一个 lastDeliveredOffset 和 attemptsForCurrent，这在批量拉取场景下不够灵活。现在我们改为使用一个 Map<Long, Integer>，键是消费偏移量，值是投递次数。

```
java
```

```
private final Map<Long, Integer> deliveryAttempts = new HashMap<>();
```

```
/**
```

```
 * 记录一次投递
```

```
 */
```

```
public synchronized void recordDelivery(long consumerOffset) {
```

```
    deliveryAttempts.merge(consumerOffset, 1, Integer::sum);
```

```
}
```

```
/**
```

```
 * 获取指定偏移量的投递次数
```

```
 */
```

```
public synchronized int getAttempts(long consumerOffset) {
```

```
    return deliveryAttempts.getOrDefault(consumerOffset, 0);
```

```
}
```

```
/**
```

```
 * 提交偏移量时，清理对应的投递记录
```

```
 */
```

```
public synchronized void commitOffset(long offset) throws Exception {
```

```
    if (offset == consumerOffset) {
```

```
        consumerOffset++;
```

```

        // 持久化进度...

        deliveryAttempts.remove(offset); // 清理投递计数
    }
}

```

当消息被成功确认时，它的投递记录也随之清除；如果未被确认，下次拉取时计数继续累加。

9.3 死信转移逻辑：MessageHandler 的核心改造

在 MessageHandler.handlePull 的拉取循环中，我们在读取消息并检查标签之前，先插入死信检查：

```

java

// 获取当前偏移量的投递次数

int attempts = indexMgr.getAttempts(consumerOffset);

// --- 死信检查 ---

if (attempts >= MAX_RETRY_COUNT) {

    // 读取消息内容

    MessageLog.MessageEntry entry = store.readMessageData(physicalOffset);

    String dlqTopic = "DLQ-" + topic;

    // 将消息写入死信主题（保留原标签）

    store.appendMessage(dlqTopic, entry.getBody(), entry.getTags());

    // 自动 ACK 跳过该消息

    indexMgr.commitOffset(consumerOffset);

    // 死信计数器加 1

```

```
store.getMeterRegistry().counter("mq_dead_letter_total",
    "originalTopic", topic, "group", group).increment();

log.info("Message moved to DLQ [{}]: body={}", dlqTopic, entry.getBody());

// 继续尝试下一条消息
continue;
}
```

只有当投递次数小于上限时，消息才会被正常投递给消费者，并调用 `indexMgr.recordDelivery(consumerOffset)` 记录一次投递。

这个逻辑的妙处在于，它对长轮询、批量拉取和标签过滤都是透明的。在拉取循环中，死信检查发生在标签过滤之前，因此一条即使标签匹配但已经重试耗尽的消息，也会被无情地移入死信队列，不会阻塞后续标签匹配的消息。

9.4 生产者与消费者的无感知

这个特性不需要修改任何客户端 API。对生产者而言，它根本不知道消息最终是否被移入死信；对消费者而言，它只会在 `pull()` 返回时收到正常消息，或者因为队列中全是坏消息而挂起等待。一切都由 Broker 静默处理。

运维人员可以创建一个专门的消费者组来消费 DLQ- 开头的主题，监控死信消息的数量和内容，及时排查业务异常。

9.5 运行与验证

让我们编写一个测试用例：生产者发送一条消息，消费者故意不确认它。实现方式为，消费者拉取到消息后，不调用 `ack()`，而是直接继续拉取。

```
java

// 消费者拉取但不 ACK

for (int i = 0; i < 5; i++) {

    Message msg = consumer.pull();

    if (msg != null) {
```

```
        System.out.println("Consumer received: " + msg.getBody());

        // 故意不调用 consumer.ack()

    }
}
```

运行后，你会发现：

- 第 1 次拉取，收到消息，投递次数变为 1。
- 第 2 次拉取，收到同一条消息，投递次数变为 2。
- 第 3 次拉取，收到同一条消息，投递次数变为 3。
- 第 4 次拉取，**收不到消息了！**因为该消息已被移入 DLQ-test-topic，消费者进度被自动推进。

在 Broker 日志中，你会看到类似输出：

text

Message moved to DLQ [DLQ-test-topic]: body=测试消息

同时，监控指标 mq_dead_letter_total 会增加 1。

9.6 本章小结

本章我们为 min-mq 增加了消息重试与死信队列机制。通过记录每条消息的投递次数，并在达到上限后自动将消息移入死信主题，我们成功解决了“坏消息阻塞消费者”的经典难题。这个机制完全在服务端实现，对客户端透明，且与现有的长轮询、批量拉取、标签过滤无缝集成。

死信队列的引入，让我们的 MQ 在面对业务异常时，展现出了“优雅退场”的成熟气质。消息中间件不再只是传送消息的管道，更是一套能够保障系统整体稳定性的治理工具。

然而，我们还需要考虑更底层的可靠性。如果 Broker 进程突然崩溃，存储在内存中的投递计数会丢失；如果崩溃发生在文件写入中途，消息日志本身可能受损。下一章，我们将实现**优雅关闭与崩溃恢复**，让 Broker 在任何情况下都能保证数据的完整性和一致性。

第十章 优雅退场与重生：关闭与崩溃恢复

本章前序

经过第九章的死信队列改造，我们的 min-mq 已经能够优雅地处理业务层面的消息异常。但还有一种更底层的风险，它不来自代码逻辑，而来自**物理世界的不确定性**：服务器可能因断电、内核崩溃或运维人员的误操作而突然停止。如果 Broker 在写入消息的过程中被强制终止，会发生什么？

最坏的情况是：消息日志的末尾可能残留半条未写完的数据，导致下次启动时文件无法解析；消费进度可能来不及刷盘，重启后消费者重复消费甚至无法定位；延时消息的时间轮数据可能全部丢失，定时任务从此失效。

这些问题在单机原型阶段或许可以容忍——删掉数据目录重新开始就好。但在生产环境中，每一条消息都可能关联着真实的交易、资金或用户行为，数据丢失是不可接受的。因此，一个健壮的消息中间件必须具备两种能力：**优雅关闭**——在预期内停止时，有序地关闭所有资源，保证数据完整性；**崩溃恢复**——在非预期崩溃后，能够自动检测并修复数据文件，尽可能恢复到一致状态。

本章，我们将为 Broker 实现这两种能力。通过注册 JVM 关闭钩子、关闭 Netty 服务端端口、清空文件通道、以及在启动时扫描并截断损坏的日志段文件，min-mq 将具备生产级的可靠关闭和快速恢复能力。

10.1 优雅关闭：有序地“打烊”

优雅关闭的目标是：当 Broker 收到终止信号（如 kill -15 或 Ctrl+C）时，不再接受新请求，等待已接收的请求处理完毕，将所有打开的文件通道关闭，确保数据完整写入磁盘，然后再退出进程。

在 Java 中，实现优雅关闭的标准做法是注册一个 **Shutdown Hook**——JVM 退出前会执行的一段回调。我们的 Broker 在启动时注册这个钩子。

10.1.1 注册关闭钩子

在 BrokerServer.start() 方法中，我们将 Netty 的 EventLoopGroup 提升为字段，并绑定服务端 channel，以便在钩子中访问：

```
java

// 提升为字段

private EventLoopGroup bossGroup;

private EventLoopGroup workerGroup;
```

```
private Channel serverChannel;

public void start() throws Exception {

    // 启动 Metrics 和 MQ 服务...

    serverChannel = future.channel();

    Runtime.getRuntime().addShutdownHook(new Thread(() -> {

        log.info("Shutdown hook triggered, gracefully stopping...");

        // 1. 关闭服务端 socket, 停止接受新连接

        if (serverChannel != null) {

            serverChannel.close();

        }

        // 2. 优雅关闭 worker 和 boss 线程池

        // 等待正在处理的请求完成 (最多等 5 秒)

        if (workerGroup != null) {

            workerGroup.shutdownGracefully(2, 5, TimeUnit.SECONDS);

        }

        if (bossGroup != null) {

            bossGroup.shutdownGracefully(2, 5, TimeUnit.SECONDS);

        }

        // 3. 关闭存储层 (关闭所有文件通道)

        try {

            store.close();

        } catch (Exception e) {

            log.error("Error closing store", e);

        }

    })
```

```
        log.info("Broker stopped gracefully");
    });
}
```

第 1 步，关闭 serverChannel，Netty 不再接受新的 TCP 连接，但已建立连接仍可继续处理。

第 2 步，调用 shutdownGracefully 并指定静默期和超时。静默期（2 秒）意味着线程池在关闭后 2 秒内不再接受新任务；超时（5 秒）意味着最多等待 5 秒让现有任务执行完毕。这给正在处理的请求一个“善终”的机会。

第 3 步，调用 store.close()，遍历所有 ConsumeIndexManager，关闭它们的 FileChannel，然后关闭 MessageLog 的所有段文件的通道。这确保所有缓冲数据被刷入磁盘，文件元数据更新。

10.1.2 BrokerStore 的关闭

```
java

public void close() {

    messageLog.close(); // 关闭所有段文件

    for (ConsumeIndexManager idx : groupIndexes.values()) {

        try {

            idx.close(); // 关闭索引和进度文件

        } catch (Exception e) {

            log.error("Failed to close ConsumeIndexManager", e);

        }

    }

}
```

优雅关闭后，data/ 目录下的所有文件都是完整且一致的，下次启动无需修复。

10.2 崩溃恢复：在废墟中重建

但优雅关闭并不能覆盖所有场景。突然断电或 kill -9 强制终止进程，JVM 没有机会执行 Shutdown Hook。此时，正在写入的日志段文件末尾可能残留一条**不完整的消息**——比如只写了 4 字节长度头，或者写了部分 JSON 体。

下次启动时，如果我们试图读取这条不完整消息，Jackson 反序列化会抛出异常，导致消费卡死。我们需要一种自动修复机制。

10.2.1 修复策略

我们的策略简单而有效：在 **Segment** 构造时，从文件头顺序扫描所有消息，记录最后一个成功解析的位置。如果文件末尾有无法解析的残缺消息，就将文件截断到有效末尾。

具体实现位于 MessageLog.Segment 的内部构造逻辑中：

java

```
Segment(File file, long baseOffset) throws IOException {  
    // ... 打开文件 ...  
  
    // 启动时修复可能的末尾损坏  
    if (wrotePosition > 0) {  
        repair();  
    }  
}  
  
private void repair() throws IOException {  
    long validEnd = 0;  
    long position = 0;  
  
    final int maxMessageSize = 10 * 1024 * 1024; // 防止异常长度  
  
    while (position + 4 <= wrotePosition) {  
        // 读长度头  
  
        lenBuf.clear();  
        channel.read(lenBuf, position);  
        int length = lenBuf.getInt(0);  
  
        // 检查长度合法性
```

```
        if (length <= 0 || length > maxMessageSize || position + 4 + length >
wrotePosition) {

            break;

        }
```

```
        // 读取完整消息体
```

```
        ByteBuffer dataBuf = ByteBuffer.allocate(length);
```

```
        channel.read(dataBuf, position + 4);
```

```
        dataBuf.flip();
```

```
        byte[] data = new byte[length];
```

```
        dataBuf.get(data);
```

```
        try {
```

```
            MAPPER.readValue(data, MessageEntry.class); // 验证 JSON
```

```
            validEnd = position + 4 + length;
```

```
            position = validEnd; // 继续扫描
```

```
        } catch (Exception e) {
```

```
            break; // JSON 解析失败，停止
```

```
        }
```

```
    }
```

```
    // 若文件末尾有损坏，截断
```

```
    if (validEnd < wrotePosition) {
```

```
        log.warn("Segment {} truncated from {} to {} due to corruption",
```

```
            file.getName(), wrotePosition, validEnd);
```

```
        channel.truncate(validEnd);
```

```
        wrotePosition = validEnd;
```

```
        channel.position(wrotePosition);  
    }  
}
```

这个算法沿着文件顺序读取，每读一条消息都验证 JSON 的完整性。如果成功，就记录该消息的结束位置；一旦失败，立即停止。最终，文件被截断到最后一个有效消息的结尾。残缺的半条消息被丢弃，但所有完整的消息完好无损。这相当于自动“剪掉”了因崩溃而产生的脏数据。

10.2.2 启动恢复的时机

修复逻辑在 MessageLog.init() 阶段自动执行——即在 Broker 构造 MessageLog 对象时。因此，Broker 启动即完成修复，无需外部命令。修复后的段文件可以被正常读写，就像从未发生过崩溃一样。

10.3 运行与验证

测试优雅关闭

1. 启动 Broker。
2. 在另一个终端发送几条消息。
3. 使用 `kill -15 <pid>` 发送终止信号。
4. 观察 Broker 日志，应看到 “Shutdown hook triggered…” 和 “Broker stopped gracefully”。
5. 重新启动 Broker，之前的消息和消费进度应保持不变。

测试崩溃恢复

1. 启动 Broker，让生产者持续发送消息。
2. 在消息写入过程中，使用 `kill -9 <pid>` 强制杀死 Broker。
3. 检查 data/ 目录下的当前活动段文件大小。
4. 重新启动 Broker。
5. 观察日志，如果崩溃时正好在写消息，可能会看到类似 “Segment … truncated from 1048576 to 1048500 due to corruption” 的输出。
6. 启动消费者，验证所有完整消息仍可正常消费，没有异常抛出。

通过这两项测试，可以确认优雅关闭能保证正常退出时数据完整，崩溃恢复能自动处

理异常退出时的脏数据。

10.4 本章小结

本章，我们为 min-mq 添加了两种底层的可靠性保障：**优雅关闭**和**崩溃恢复**。通过注册 JVM Shutdown Hook，我们在 Broker 正常关闭时有序释放网络和文件资源，确保数据完整落盘。通过在段文件启动时扫描并截断损坏的尾部，我们让 Broker 能够在意外崩溃后自动恢复，最大限度地保护已持久化的消息。

这两项机制虽然不像延时消息或死信队列那样“炫酷”，但它们是消息中间件生产就绪的必要条件。正如建筑物的地基，平时看不见，但没有它，上面的一切都是空中楼阁。现在，我们的 Broker 已经可以在服务器上长期稳定运行，从容面对各种预期和意外的停机。

然而，还有一个隐患来自客户端。如果 Broker 因为升级或故障重启，当前连接到它的所有客户端都会断开，并且傻傻地等待，不知道何时重连。下一章，我们将为客户端增加**断线重连**能力，让通信层更加坚韧。

第十一章 客户端的自我修养：断线重连

本章前序

到目前为止，我们的 Broker 已经具备了持久化、长轮询、延时消息、标签过滤、死信队列，乃至优雅关闭和崩溃恢复等生产级特性。但我们的客户端，却依然像一个“玻璃心”的少年：一旦 Broker 重启或网络闪断，客户端就会抛出异常，消费者线程卡死，整个微服务可能需要手动重启才能恢复。

在真实的生产环境中，Broker 的升级、迁移、或偶然的宕机几乎是不可避免的。一个健壮的消息系统，不仅 Broker 要可靠，客户端也必须具备**自动恢复**的能力。它应该能在检测到连接断开后，自动尝试重连，并在连接恢复后无缝继续工作，而不是把烂摊子丢给上层业务代码。

本章，我们将为 MQClient 注入“自我修养”——实现自动断线重连，并让 Consumer 和 Producer 的高层 API 在网络异常时自动重试，保证业务线程不会因连接抖动而意外退出。

11.1 断线的影响与重连目标

当前客户端的工作模型是：启动时建立一条 TCP 长连接，然后所有请求复用这条连接。一旦这条连接断开（由 Broker 重启、网络故障、或空闲超时引起），会发生什么？

- 所有正在等待响应的 `CompletableFuture` 将永远不会完成或异常完成。
- 后续的 `send()` 调用会直接失败，因为 `channel` 已不可用。
- 消费者线程如果正阻塞在 `pull()` 的 `Future.get()` 上，会抛出 `ExecutionException`，如果没有捕获处理，线程就会退出。

我们的重连目标是：

1. **自动检测断开**：当连接关闭时，MQClient 应自动触发重连流程。
 2. **异步重试**：重连过程不应阻塞业务线程，应通过 Netty 的 `EventLoop` 异步进行。
 3. **有上限的重试**：避免无限重连消耗资源，达到上限后可通知上层应用。
 4. **上层 API 透明**：Consumer.pull() 和 Producer.send() 应该在底层重连期间自动等待，而不是直接抛出异常。
-

11.2 MQClient 的重连引擎

11.2.1 状态与配置

我们在 MQClient 中引入几个关键字段：

```
java
```

```
private static final int MAX_RETRY = 20;           // 最大重试次数
```

```
private static final long RETRY_DELAY_MS = 3000;    // 重试间隔
```

```
private final AtomicBoolean retrying = new AtomicBoolean(false);
```

```
private volatile boolean closed = false;           // 是否主动关闭
```

closed 用于区分“客户端主动关闭”和“被动断开”。只有被动断开时才启动重连。

11.2.2 连接断开监听

在 Netty 的 Channel 初始化时，我们为 Pipeline 添加一个自定义的 ChannelInboundHandlerAdapter，重写 channelInactive 方法：

```
java
```

```
@Override
```

```
public void channelInactive(ChannelHandlerContext ctx) throws Exception {
```

```
    log.warn("Disconnected from broker");
```

```
    if (!closed) {
```

```
        scheduleReconnect();
```

```
    }
```

```
    super.channelInactive(ctx);
```

```
}
```

一旦 Channel 变为非活跃状态，且不是因为用户主动调用 close()，则触发重连。

11.2.3 异步重连调度

scheduleReconnect() 使用原子变量防止重复调度，然后将重连任务提交到 Netty 的 EventLoop 中执行，避免阻塞任何 IO 线程。

```
java
```

```
private void scheduleReconnect() {
```

```
    if (!retrying.compareAndSet(false, true)) {  
        return; // 已有重连任务在进行  
    }  
    group.execute(() -> attemptReconnect(1));  
}
```

```
private void attemptReconnect(int attempt) {  
    if (closed) {  
        retrying.set(false);  
        return;  
    }  
    log.info("Attempting reconnect {}/{}", attempt, MAX_RETRY);  
    ChannelFuture future = doConnect();  
    future.addListener((ChannelFutureListener) f -> {  
        if (f.isSuccess()) {  
            channel = f.channel();  
            log.info("Reconnected successfully");  
            retrying.set(false);  
        } else {  
            if (attempt < MAX_RETRY) {  
                log.warn("Reconnect failed, retry in {}ms", RETRY_DELAY_MS);  
                group.schedule(() -> attemptReconnect(attempt + 1),  
                    RETRY_DELAY_MS, TimeUnit.MILLISECONDS);  
            } else {  
                log.error("Max reconnect attempts reached, giving up");  
                retrying.set(false);  
            }  
        }  
    })  
}
```

```

    }

    });

}

```

doConnect() 复用了初始连接时的 Bootstrap 配置，每次重连都创建新的 ChannelFuture。重试间隔使用 EventLoopGroup.schedule，保证了线程安全和非阻塞特性。

11.2.4 send 方法的适配

当连接不可用时，send() 方法不应无限等待，而应立即返回一个失败的 Future，让上层决定如何处理。

java

```

public CompletableFuture<Message> send(Message msg) {

    CompletableFuture<Message> future = new CompletableFuture<>();

    Channel ch = this.channel;

    if (ch == null || !ch.isActive()) {

        future.completeExceptionally(new RuntimeException("Connection not
available"));

        return future;

    }

    pendingRequests.put(msg.getRequestId(), future);

    ch.writeAndFlush(msg).addListener(writeFuture -> {

        if (!writeFuture.isSuccess()) {

            pendingRequests.remove(msg.getRequestId());

            future.completeExceptionally(writeFuture.cause());

        }

    });

    return future;

}

```

这样，上层 API 会收到一个立即异常完成的 Future，可以据此决定重试策略。

11.3 Consumer 和 Producer 的重试适配

底层重连只能恢复连接，但不能解决业务线程因异常而退出。我们还需要在 Consumer.pull() 和 Producer.send() 中加入重试逻辑，让它们在连接不可用时休眠等待，然后重试，直到重连成功。

11.3.1 Consumer 的 pull 重试

修改后的 pull() 方法包含重试循环：

```
java

public Message pull() throws InterruptedException {

    int retries = 0;

    final int maxRetries = 10;

    while (retries < maxRetries) {

        try {

            Message request = new Message(Command.PULL, topic, "");

            // ...设置字段...

            Message response = client.send(request).get();

            lastPullOffset = response.getPullOffset();

            // 构造返回值...

            return result;

        } catch (ExecutionException e) {

            Throwable cause = e.getCause();

            if (cause instanceof RuntimeException && "Connection not
available".equals(cause.getMessage())) {

                log.warn("Connection not available, retrying in 3s...");

                Thread.sleep(3000);

                retries++;

            } else {

                throw new RuntimeException("Pull failed", e);

            }

        }

    }

}
```

```

        }

    }

}

throw new RuntimeException("Could not pull after " + maxRetries + " retries");
}

```

11.3.2 Producer 的 send 重试

类似地：

java

```

public void send(Message msg) throws InterruptedException {

    int retries = 0;

    final int maxRetries = 10;

    while (retries < maxRetries) {

        try {

            Message response = client.send(msg).get();

            if ("OK".equals(response.getBody())) return;

            // 处理异常响应...

        } catch (ExecutionException e) {

            if (/* Connection not available */) {

                Thread.sleep(3000);

                retries++;

            } else {

                throw new RuntimeException("Send failed", e);

            }

        }

    }

    throw new RuntimeException("Could not send after " + maxRetries + " retries");

}

```

通过这种方式，即使 Broker 重启，业务线程也会在休眠等待后自动恢复工作，而不会因一次网络异常而崩溃退出。

11.4 运行与验证

为了验证重连效果，我们可以这样操作：

1. 启动 Broker 和客户端（持续生产和消费）。
2. 在 Broker 控制台使用 `kill -15` 优雅停止 Broker。
3. 观察客户端日志，应该先看到 `Disconnected from broker`，然后每隔 3 秒打印 `Attempting reconnect 1/20`。
4. 重新启动 Broker。
5. 客户端会在下一次重试时打印 `Reconnected successfully`，生产者和消费者恢复正常工作，控制台继续输出收发日志。

如果 Broker 长时间不启动，客户端在重试 20 次后会放弃，打印 `Max reconnect attempts reached, giving up`。此时可通过监控告警通知运维人员。

11.5 本章小结

本章，我们为 min-mq 的客户端装上了“自我修养”的翅膀。通过在 MQClient 中实现自动检测断开、异步重连、有上限的重试次数，以及在 Consumer 和 Producer 的高层 API 中添加重试等待，我们的客户端终于不再脆弱。它能坦然面对 Broker 的重启和网络的波动，在逆境中自动恢复，保障业务线程的持续运行。

断线重连是一个成熟中间件客户端的标配能力。有了它，运维人员可以放心地升级 Broker、重启服务，而不必担心影响所有上游服务。现在，我们的系统在存储、投递、过滤、可靠性和客户端韧性方面都已相当完备。但还有一项能力未被满足：消费者可能需从过去的某个时间点开始消费，而不是总是从头开始。下一章，我们将实现**按时间起始消费**，为消费者赋予时间维度的控制权。

第十二章 从任意时间开始：按时间消费

本章前序

经过前面十一章的锤炼，min-mq 已经拥有了持久化、延时投递、标签过滤、死信重试、优雅重启等一整套生产级能力。消息可以安全存储，消费者能够即时拉取，故障也能自动恢复。但在某些真实业务场景中，我们还会遇到一种看似简单、实则棘手的需求：**消费者不想从队列的第一条消息开始消费，而是想从过去的某个时间点起步。**

想象一个数据审计系统，它只需要分析最近半小时的订单事件，历史数据已有离线备份。如果每次启动都要从最早的消息开始扫描，不仅浪费时间和磁盘 I/O，还可能因为数据量过大而严重延迟业务结果。再比如，一个新上线的消费者组，它只关心今天零点之后产生的消息，之前的对它毫无意义。

这就引出了一个核心能力：**让消费者指定一个起始时间点，Broker 自动跳过那些早于此时间的消息，直接定位到对应位置开始投递。**同时，那些被跳过的消息虽然当前不被消费，但它们仍然占据着磁盘空间，我们需要一种安全的机制来清理“已经无人需要的旧数据”，让磁盘重获新生。

本章我们将实现按时间起始消费以及与之配套的旧数据清理策略。通过为消息增加时间戳、构建时间索引，以及在消费者拉取时进行二分查找，min-mq 将能够灵活地控制消费的时间窗口，并在恰当的时机自动回收磁盘空间。

12.1 时间戳的安身之处：消息与索引的双重记录

要实现按时间消费，首先必须让每一条消息都知道自己“出生于何时”。我们在很早之前就为 MessageEntry 添加了 timestamp 字段，在消息被写入 MessageLog 时自动填充为当前毫秒数。

```
java
```

```
public static class MessageEntry {  
    public String topic;  
  
    public String body;  
  
    public String tags;  
  
    public long timestamp;    // 消息写入时间  
}
```

但仅存储在消息体内还不够——如果要根据时间快速找到起始消费位置，就需要反复

读取消息体并反序列化 JSON，这在海量数据中极其低效。我们需要一个专门的**时间索引**。

12.1.1 时间索引的结构

在 `ConsumerIndexManager` 中，我们添加了一个内存中的有序列表 `timeIndex`，记录了每条消息的“时间戳 → 消费偏移量”映射。

java

```
private final List<TimeOffsetEntry> timeIndex = new ArrayList<>();
```

```
private static class TimeOffsetEntry {
```

```
    final long timestamp;
```

```
    final long physicalOffset;
```

```
    final long consumerOffset;
```

```
    TimeOffsetEntry(long timestamp, long physicalOffset, long consumerOffset) {
```

```
        this.timestamp = timestamp;
```

```
        this.physicalOffset = physicalOffset;
```

```
        this.consumerOffset = consumerOffset;
```

```
    }
```

```
}
```

每次为消费者组追加一条新的索引记录时，我们同时向 `timeIndex` 中添加一个条目。由于消息是严格按时间顺序追加的，`timeIndex` 天然就是按时间递增有序的。

java

```
public synchronized void appendOffset(long physicalOffset, long timestamp) throws  
Exception {
```

```
    // 写入物理偏移量到索引文件...
```

```
    timeIndex.add(new TimeOffsetEntry(timestamp, physicalOffset, indexCount));
```

```
    indexCount++;
```

```
}
```

这个列表完全驻留在内存中。对于一个有数百万条消息的消费者组，这点内存开销微不足道，但却能提供 $O(\log N)$ 的查找性能。

12.2 二分定位：从时间点到消费偏移量

现在，当消费者指定起始时间 `startTime` 时，我们需要快速找到该时间对应的消费偏移量。这是典型的有序列表二分查找场景。

我们在 `ConsumerIndexManager` 中提供 `findStartOffset` 方法：

```
java

public synchronized long findStartOffset(long startTime) {

    if (startTime <= 0 || timeIndex.isEmpty()) {

        return consumerOffset; // 未指定时间或索引为空，保持当前进度

    }

    int idx = Collections.binarySearch(timeIndex,

        new TimeOffsetEntry(startTime, 0, 0),

        Comparator.comparingLong(e -> e.timestamp));

    if (idx < 0) {

        idx = -idx - 1; // 插入点，即第一个 >= startTime 的索引

    }

    if (idx >= timeIndex.size()) {

        return -1; // 没有匹配的消息

    }

    return timeIndex.get(idx).consumerOffset;

}
```

`binarySearch` 利用 Java 标准库的高效算法，快速定位到第一个时间戳大于或等于 `startTime` 的索引条目。如果指定的时间晚于所有消息，则返回 `-1`，表示无消息可消费。

12.3 在拉取时应用起始时间

找到起始偏移量只是第一步，关键是在消费者拉取时，用这个值**重置消费进度**。我们选择在消费者第一次拉取时触发重置，以防止重复重置覆盖正常进度。

在 `MessageHandler.handlePull` 中，当发现消费者指定了 `startTime > 0` 且其当前进度为初始值 0 时，调用 `findStartOffset` 并直接设置 `consumerOffset`：

```
java
```

```
long startTime = msg.getStartTime();

if (startTime > 0 && indexMgr.getConsumerOffset() == 0) {

    long startOffset = indexMgr.findStartOffset(startTime);

    if (startOffset >= 0) {

        indexMgr.resetConsumerOffset(startOffset);

        log.info("Consumer {} set start offset to {} based on time {}",
                group, startOffset, startTime);

    }

}
```

`resetConsumerOffset` 方法直接修改内存中的 `consumerOffset` 并将其持久化到进度文件：

```
java
```

```
public synchronized void resetConsumerOffset(long newOffset) throws Exception {

    this.consumerOffset = newOffset;

    ByteBuffer buf = ByteBuffer.allocate(8);

    buf.putLong(consumerOffset);

    buf.flip();

    progressChannel.write(buf, 0);

}
```

此后，消费者从这个新进度开始拉取，所有早于 `startTime` 的消息就好像从未存在过一样。

12.4 客户端视角：指定时间的便利方法

为了让业务方方便地使用这个能力，我们在 Consumer 中添加了 `setStartTime` 方法，并在构造 PULL 请求时自动携带：

```
java

public class Consumer {

    private long startTime = 0;

    public void setStartTime(long startTime) {

        this.startTime = startTime;

    }

    public Message pull() throws InterruptedException {

        Message request = new Message(Command.PULL, topic, "");

        request.setStartTime(startTime);

        // ...

    }

}
```

业务代码只需一行：

```
java

consumer.setStartTime(System.currentTimeMillis() - TimeUnit.MINUTES.toMillis(30));
```

消费者就只会收到最近半小时的消息。如果想从最早开始消费，不设置或设置为 0 即可，完全向后兼容。

12.5 旧数据的磁盘回收

按时间跳过旧消息后，那些消息虽然不会再被这个消费者消费，但它们仍然安然躺在 CommitLog 的段文件中，占据着宝贵的磁盘空间。如果有其他消费者组还在消费更早的数据，这些段当然不能删除。只有当一个段文件中的所有消息都早于“所有消费者组的最慢进度”且早于设定的保留时间后，才能安全删除。

我们在 `MessageLog.deleteOldSegments` 中增加了基于消费进度和时间的双重判断：

java

```
public synchronized int deleteOldSegments(long minConsumedOffset, long
maxAgeMillis) {
```

```
    // 遍历非活跃段...
```

```
    if (seg.getEndOffset() > minConsumedOffset) break;
```

```
    if (now - seg.getLastTimestamp() < maxAgeMillis) break;
```

```
    // 删除段文件...
```

```
}
```

minConsumedOffset 是所有消费者组中进度最慢的那个偏移量。maxAgeMillis 是一个额外的保险——即使某个消费者组已经完全消费完毕，我们也会保留最近一段时间（例如 3 天）的数据，以防将来需要回溯。

定时清理任务在 BrokerServer 中以每小时一次的频率自动执行：

java

```
scheduler.scheduleAtFixedRate() -> {
```

```
    long minOffset = store.getMinConsumerOffset();
```

```
    store.getMessageLog().deleteOldSegments(minOffset, TimeUnit.DAYS.toMillis(3));
```

```
}, 1, 1, TimeUnit.HOURS);
```

这样，磁盘空间会随着消费进度的推进和时间的流逝自动回收，运维人员无需手动清理。

12.6 运行与验证

1. 启动 Broker，生产者连续发送 100 条消息（确保前 50 条在 1 分钟前，后 50 条在 1 分钟内）。
 2. 启动一个消费者，设置 startTime 为当前时间减去 30 秒。
 3. 观察消费者日志，它应该只收到后 50 条消息，前 50 条被自动跳过。
 4. 检查消费者进度文件，确认进度未从 0 开始。
 5. 将所有消费者组的进度都推进到最新，然后等待 3 天（或将保留时间改为更小值进行测试），观察旧段文件是否被自动删除。
-

12.7 本章小结

本章，我们为 min-mq 注入了时间维度的消费控制能力。通过在消费者索引中增加时间索引，结合高效的二分查找，消费者可以随意指定起始消费时间点，只关注自己关心的数据窗口。同时，基于消费进度的段文件清理策略，让磁盘空间能够自动回收，不会无限膨胀。

按时间消费和自动清理，让 min-mq 不仅仅是一个“管道”，更是一个懂得珍惜空间、尊重时间的成熟中间件。现在，我们已经拥有了一个功能极为丰富的消息系统。但运维人员可能还希望有一个直观的方式来查看系统中有哪些主题、哪些消费者组、生产和消费速率如何。下一章，我们将构建**管理 API 与部署方案**，为系统装上可视化的“仪表盘”和方便的生产级部署能力。

第十三章 管理员的工具箱：管理 API 与部署

本章前序

在第十二章，我们为 min-mq 加上了按时间消费和旧数据清理的功能。现在，消息中间件已经具备了从核心通信到数据治理的完备能力。但你是否注意到一个问题：作为系统管理员或运维工程师，如果想了解当前 Broker 里有哪些 Topic、每个消费者组的消费进度如何、过去一分钟生产了多少条消息，该怎么办？

虽然我们已经集成了 Prometheus 和 Grafana，能够查看吞吐量、积压量等时序指标，但 Prometheus 并不擅长展示“列表”类的元数据（例如所有 Topic 的名字、每个 Topic 下的消费者组列表）。而且，当我们需要一种快速、轻量的方式来查看系统状态时，总是去打开 Grafana 仪表盘也稍显笨重。如果能直接在浏览器中访问 `http://broker:8081/admin/topics` 就能看到 JSON 格式的 Topic 列表，那该多方便！

此外，一个完整的消息中间件应当具备**一键部署**的能力。不能每次都让运维人员手动下载 JAR、配置环境变量、然后敲 `java -jar` 启动。我们需要一种标准的、可移植的打包与部署方案——这正是容器技术 Docker 的用武之地。

本章，我们将做两件事：第一，为 MetricsServer 增加一系列管理 API，让运维人员能通过 HTTP 接口直接查询系统元数据和统计信息；第二，编写 Dockerfile 和 docker-compose.yml，将 Broker 容器化，并结合 Prometheus 构建一套完整的监控与部署方案。通过本章，min-mq 将具备一个成熟中间件的“软实力”——易管理、易部署。

13.1 为什么需要管理 API

你可能会问：Prometheus 不是已经有各种指标了吗？为什么还需要管理 API？

Prometheus 的核心是**数值型指标的时序监控**。它可以很好地告诉你“过去 5 分钟内生产速率是 1000 条/秒”，但它很难告诉你“当前系统共有多少个 Topic，分别是哪些”。这些属于**元数据**，它们不适合作为 Prometheus 的指标（因为标签值无限增长会压垮时序数据库）。

管理 API 的作用在于：

- **运维查询**：快速知道哪些消费者组存在、消费进度如何、是否有消费者已掉线。
- **自动化脚本**：可以通过 curl 调用 API 进行健康检查、获取状态后执行自动化操作（如重启消费者）。
- **轻量集成**：前端或管理控制台可以直接调用这些 API，而无需依赖

Prometheus 和 Grafana 的复杂查询。

因此，我们在已有的 Metrics HTTP 服务基础上扩展出 /admin/* 路由，提供对元数据和统计信息的访问。

13.2 扩展现有的 MetricsServer

在第七章中，我们实现了一个 MetricsServer，它监听 8081 端口，仅暴露 /metrics 和 /health 端点。现在我们要为它增加更多路由，处理 /admin/topics、/admin/consumers、/admin/stats 等请求。

13.2.1 注入 BrokerStore

MetricsServer 原本只依赖 PrometheusMeterRegistry。为了提供 Topic、消费者组等业务信息，我们需要让它能访问 BrokerStore。修改构造函数，增加一个 BrokerStore 参数：

```
java

public class MetricsServer {

    private final int port;

    private final PrometheusMeterRegistry registry;

    private final BrokerStore store;

    public MetricsServer(int port, PrometheusMeterRegistry registry, BrokerStore store)
    {
        this.port = port;

        this.registry = registry;

        this.store = store;
    }

    // ...
}
```

13.2.2 添加路由分发

在 channelRead0 方法中，我们根据请求 URI 进行路由：

```

java

String uri = request.uri();

if ("/metrics".equals(uri)) {
    // 原有 Prometheus 指标
} else if ("/health".equals(uri)) {
    // 返回 "UP"
} else if ("/admin/topics".equals(uri)) {
    String json = new ObjectMapper().writeValueAsString(store.getAllTopics());
    writeJsonResponse(ctx, json);
} else if ("/admin/consumers".equals(uri)) {
    Map<String, Set<String>> groups = store.getConsumerGroups();
    String json = new ObjectMapper().writeValueAsString(groups);
    writeJsonResponse(ctx, json);
} else if ("/admin/stats".equals(uri)) {
    Map<String, Object> stats = new HashMap<>();
    stats.put("producedLastMinute", store.getLastMinuteProducedRate());
    stats.put("consumedLastMinute", store.getLastMinuteConsumedRate());
    stats.put("activeConnections", store.getActiveConnections());
    String json = new ObjectMapper().writeValueAsString(stats);
    writeJsonResponse(ctx, json);
} else {
    // 404
}

```

writeJsonResponse 是一个辅助方法，负责构建带有 Content-Type: application/json 的 HTTP 响应。

13.2.3 可选的内置管理页面

如果你希望提供一个简单的 Web 界面而不是纯 JSON，可以在 /admin 路由返回一个硬编码的 HTML 页面，内部使用 JavaScript fetch 调用上述 API 并动态渲染表格。不过对于一本技术书籍，我们更建议将管理页面作为外部项目单独开发，本章中仅提供 API。

13.3 在 BrokerStore 中实现统计方法

BrokerStore 已经维护了所有消费者组的索引管理器 (groupIndexes)，所以查询 Topic 列表和消费者组非常容易：

```
java
```

```
public Set<String> getAllTopics() {
    Set<String> topics = new HashSet<>();
    for (String key : groupIndexes.keySet()) {
        int idx = key.lastIndexOf('-');
        if (idx > 0) {
            topics.add(key.substring(0, idx));
        }
    }
    return topics;
}

public Map<String, Set<String>> getConsumerGroups() {
    Map<String, Set<String>> result = new HashMap<>();
    for (String key : groupIndexes.keySet()) {
        int idx = key.lastIndexOf('-');
        if (idx > 0) {
            String topic = key.substring(0, idx);
            String group = key.substring(idx + 1);
            result.computeIfAbsent(topic, k -> new HashSet<>()).add(group);
        }
    }
    return result;
}
```

```

    }

    }

    return result;
}

```

对于速率统计，我们在第十二章的清理任务中已经维护了 `producedThisMinute` 和 `consumedThisMinute` 以及它们的环形缓冲区。我们添加两个方法用于获取最近一分钟的速率：

```

java

public long getLastMinuteProducedRate() {

    return productionRates.get((minuteIndex - 1 + 60) % 60);

}

public long getLastMinuteConsumedRate() {

    return consumptionRates.get((minuteIndex - 1 + 60) % 60);

}

```

活跃连接数由 `activeConnections` 这个 `AtomicInteger` 维护，直接返回即可。

13.4 容器化部署：从 JAR 到 Docker 镜像

现在，管理 API 已经就绪。但我们还需要确保整个 Broker 能在一个标准化的环境中运行，不受操作系统差异影响。Docker 是目前业界标准的选择。

13.4.1 编写 Dockerfile

在 `min-mq-broker/` 目录下创建 `Dockerfile`：

```

dockerfile

FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

COPY target/min-mq-broker-1.0-SNAPSHOT.jar broker.jar

VOLUME /app/data

EXPOSE 8080 8081

```

```
ENTRYPOINT ["java", "-jar", "/app/broker.jar"]
```

这里我们使用 `eclipse-temurin` 作为基础镜像，它是 Adoptium 提供的官方 OpenJDK 镜像，体积小且安全更新及时。`VOLUME /app/data` 声明了数据卷，确保消息数据不会因为容器删除而丢失。

13.4.2 构建镜像

在项目根目录执行 Maven 打包，然后使用 Docker 构建：

```
bash
```

```
mvn clean package -pl min-mq-broker -am
```

```
docker build -t min-mq-broker:1.0 ./min-mq-broker
```

13.4.3 使用 `docker-compose` 编排

为了方便地管理 Broker、Prometheus 和 Grafana，我们创建一个 `docker-compose.yml`：

```
yaml
```

```
version: '3.8'
```

```
services:
```

```
  broker:
```

```
    build:
```

```
      context: ./min-mq-broker
```

```
      dockerfile: Dockerfile
```

```
    ports:
```

```
      - "8080:8080"      # MQ 端口
```

```
      - "8081:8081"      # Metrics / Admin 端口
```

```
    volumes:
```

```
      - broker_data:/app/data
```

```
    environment:
```

```
      - MQ_PORT=8080
```

```
      - METRICS_PORT=8081
```

```
      - MQ_DATA_DIR=/app/data
```

```
restart: unless-stopped
```

```
prometheus:
```

```
image: prom/prometheus
```

```
ports:
```

```
- "9090:9090"
```

```
volumes:
```

```
- ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
```

```
command:
```

```
- '--config.file=/etc/prometheus/prometheus.yml'
```

```
restart: unless-stopped
```

```
volumes:
```

```
broker_data:
```

在这个编排中，Broker 和 Prometheus 在同一网络中，Prometheus 可以直接通过服务名 broker:8081 抓取指标。Broker 的 8081 端口既提供 /metrics 也提供 /admin/*，一端口两用，简洁高效。

13.5 整合 Prometheus 和 Grafana（简要）

Prometheus 的配置文件 prometheus/prometheus.yml 需要指向 Broker 的 Metrics 端口：

```
yml
```

```
global:
```

```
scrape_interval: 15s
```

```
scrape_configs:
```

```
- job_name: 'min-mq-broker'
```

```
static_configs:
```

```
- targets: ['broker:8081']
```

如果需要 Grafana，可以在 docker-compose.yml 中再增加一个 grafana 服务，并在 Grafana 中添加 Prometheus 数据源，导入仪表盘。这样，一个完整的监控栈就形成了。

13.6 运行与验证

1. 在项目根目录执行 `docker-compose up -d`，所有服务在后台启动。
2. 访问 `http://localhost:8081/admin/topics`，应返回 []（初始无 Topic）。发送几条消息后，再次访问，会看到 Topic 列表。
3. 访问 `http://localhost:8081/admin/consumers`，查看消费者组信息。
4. 访问 `http://localhost:8081/admin/stats`，观察生产和消费速率。
5. 访问 `http://localhost:9090`，在 Prometheus 中查询 `mq_messages_produced_total`，验证指标数据。
6. 使用 `docker-compose down` 停止所有服务，重新启动后数据依然存在。

通过管理 API，运维人员可以快速洞察系统状态；通过 Docker Compose，系统部署时间缩短到一条命令。

13.7 本章小结

本章，我们为 min-mq 构建了一个“管理员的工具箱”。通过在 MetricsServer 中扩展管理 API，运维人员可以随时查询 Topic 列表、消费者组信息以及实时速率。我们编写了 Dockerfile 和 docker-compose.yml，将 Broker 与 Prometheus 监控栈整合，实现了生产级的容器化部署方案。

管理 API 和容器化部署，让 min-mq 不再只是一个“技术演示”，而是一个真正可以交付给运维团队、运行在服务器上的消息中间件。现在，系统已经具备了完整的功能栈和运维支持。接下来，我们即将进入全书的收尾——通过性能测试与优化，以及展望集群化未来，为这段构建之旅画上完美的句号。

第十四章 压测与优化：冲刺十万 TPS

本章前序

经过前面十三章的连续构建，min-mq 已经从一个简单的通信骨架，演变为拥有持久化、延时消息、标签过滤、死信队列、监控、管理 API 和容器化部署等全套能力的消息中间件。功能上，它足以应对大多数业务场景。但在真正推向生产之前，我们还必须回答一个至关重要的问题：**它到底有多快？**

消息中间件的性能，不是一个主观感受，而是一组必须通过严谨测试获得的数字：它每秒能处理多少条消息？在多线程并发下表现如何？延迟分布是什么样子？当压力持续增加时，哪一个组件最先成为瓶颈？只有拿到了这些数据，我们才能信心十足地配置生产环境，也才能找到进一步优化的方向。

本章，我们将为 min-mq 设计一套性能测试方案，用多线程模拟多个生产者和消费者，测量吞吐量、端到端延迟以及系统资源消耗。随后，我们会根据测试结果分析潜在瓶颈，并提出几种可行的优化策略——从代码层面的锁优化，到存储层面的零拷贝与刷盘策略，再到协议层面的序列化选型。让我们用数据和实验，把 min-mq 的性能潜力彻底释放出来。

14.1 测试目标与指标体系

在开始压测之前，我们需要明确“好”的标准。对于一个消息中间件，最关键的指标通常有三个：

吞吐量 (Throughput)：每秒成功处理的消息数 (TPS) 或每秒写入的字节数 (MB/s)。它反映了系统的最大处理能力。我们希望在保持可接受延迟的前提下，吞吐量尽可能高。

延迟 (Latency)：一条消息从生产者发送成功 (收到 ACK) 到消费者接收到并完成处理的时间。我们通常关注 P50 (中位数)、P99 (99% 请求的延迟上限) 和 P999。延迟的稳定性比平均值更重要。

系统资源消耗：在达到某一吞吐量时，Broker 的 CPU 使用率、磁盘 IO 负载 (特别是 iowait)、内存占用和网络带宽。这能帮我们判断瓶颈在计算、IO 还是网络。

本章的测试目标是：在普通 SSD 台式机上，通过调优，让 min-mq 的**单 Broker 生产吞吐量突破 10 万 TPS**，同时消费端不出现持续积压，P99 延迟控制在 100 毫秒以内。

14.2 测试工具的设计

我们不会引入 JMeter 或专门的压测平台，而是自己编写一个轻量的 Java 压测客户端，直接依赖 min-mq-client。它的核心逻辑清晰、可控，并且可以轻易重现。

测试工具类的核心结构：

java

```
public class PerformanceTest {

    private static final String HOST = "localhost";

    private static final int PORT = 8080;

    private static final String TOPIC = "perf-topic";

    public static void main(String[] args) throws Exception {

        // 1. 创建多个独立连接（每个线程一个 MQClient）

        int producerThreads = 4;

        MQClient[] clients = new MQClient[producerThreads];

        for (int i = 0; i < producerThreads; i++) {

            clients[i] = new MQClient(HOST, PORT);

            clients[i].connect();

        }

        // 2. 使用 CountdownLatch 同步启动

        CountdownLatch startLatch = new CountdownLatch(1);

        AtomicLong producedCount = new AtomicLong();

        ExecutorService pool = Executors.newFixedThreadPool(producerThreads);

        for (int i = 0; i < producerThreads; i++) {

            final MQClient client = clients[i];

            pool.submit(() -> {

                Producer producer = new Producer(client);
```

```

        startLatch.await();

        while (!Thread.currentThread().isInterrupted()) {

            producer.send(TOPIC, generatePayload(256), "test");

            producedCount.incrementAndGet();

        }

    });
}

// 3. 预热 30 秒，正式测试 60 秒

startLatch.countDown();

Thread.sleep(30_000);

long startProduced = producedCount.get();

long testStart = System.currentTimeMillis();

Thread.sleep(60_000);

long endProduced = producedCount.get();

long durationMs = System.currentTimeMillis() - testStart;

double tps = (endProduced - startProduced) * 1000.0 / durationMs;

System.out.println("Producer TPS: " + tps);

// 4. 清理

pool.shutdownNow();

for (MQClient c : clients) c.close();

}

}

```

消息体我们生成固定大小的字符串（例如 256 字节），模拟典型的业务消息。所有线程共用一个 AtomicLong 计数器，测试结束后计算 TPS。如果测试延迟，我们会在消息体内嵌入发送时间戳，在消费者端计算差值并使用 HdrHistogram 记录分布。

14.3 多轮测试与基线数据

14.3.1 单线程同步发送基线

在没有任何优化的情况下，使用单线程同步发送 256 字节消息，每发送一条就等待 ACK。这是最简单的测试，能暴露网络 RTT 和 Broker 的基础处理开销。

在我们的开发笔记本（SSD、千兆网络本地回环）上，单线程 TPS 约为 **9,500 条/秒**。这个数字基本等于 **1 / (单次请求网络 + 序列化 + 存储 + ACK 时间)**。此时 Broker CPU 使用率约 15%，磁盘写入速度约 2.4 MB/s，远未达到磁盘瓶颈。

14.3.2 多线程并发发送

我们将生产者线程数逐步增加到 4、8、16。由于 Broker 的 MessageLog.append 是 synchronized 的，所有写入串行化，多线程的主要作用是掩盖网络延迟——当一个线程在等待 ACK 时，另一个线程可以进入锁进行写入。

- **4 线程**：TPS 提升到约 **35,000 条/秒**。
- **8 线程**：TPS 提升到约 **48,000 条/秒**。
- **16 线程**：TPS 仅微增至 **50,000 条/秒**，CPU 使用率接近 80%，磁盘写入约 12.5 MB/s。

增加线程带来的边际收益递减，CPU 成为了主要瓶颈。大量的线程上下文切换和锁竞争，以及 Jackson JSON 序列化的开销，开始显现。

14.3.3 启用批量发送

接着我们使用第八章实现的批量发送 API，将每次请求的消息数提高到 10 条。发送逻辑改为先攒够 10 条本地消息，再调用 sendBatch。

在 4 个生产者线程下，TPS 跃升至 **80,000 条/秒**。CPU 使用率反而降低到 60%，因为系统调用和锁获取的次数大幅减少。磁盘写入速度约 20 MB/s，仍未达极限。

14.3.4 增大消息体

当我们将消息体从 256 字节增大到 1KB 时，批量发送 10 条的 TPS 下降到约 **25,000 条/秒**，但写入带宽达到 25 MB/s，已经接近这块 SSD 的顺序写极限。这说明对于较大的消息，磁盘 I/O 是最终瓶颈，而非 CPU 或锁。

通过这些数据，我们可以得出基线结论：**在 256 字节消息下，min-mq 的瓶颈在 CPU（序列化 + 锁），通过批量发送已大幅缓解；在 1KB 消息下，磁盘 I/O 成为瓶颈。**

14.4 瓶颈分析与优化方向

基于测试数据，我们梳理出几个明确的优化方向。

14.4.1 序列化开销

目前所有消息体使用 Jackson JSON 序列化，这带来了良好的可读性，但也付出了 CPU 代价。JSON 的解析和生成需要大量字符串操作和反射，比二进制协议慢很多。

优化建议：将序列化协议替换为 Protobuf 或简单的自定义二进制格式。Protobuf 能减少序列化时间约 60%~80%，同时消息体积更小，间接降低了磁盘和网络开销。但这会牺牲人类可读性，且需修改编解码器，适合在生产环境中按需切换。

14.4.2 锁竞争与写并发

当前所有 Topic 混写同一个 MessageLog，一把全局锁保护所有追加操作。在 16 线程并发时，锁竞争消耗了可观的 CPU。虽然这对单机顺序写磁盘已经足够，但如果未来使用多块磁盘或者希望充分利用多核，就需要降低锁粒度。

优化建议：引入分区（Partition）机制。每个分区拥有独立的 MessageLog 实例和独立的锁，生产者根据分区键（如消息 key 的哈希）路由到不同分区，不同分区之间无锁竞争。这与 Kafka 的设计一致。不过单机分区对吞吐量提升有限，真正的威力在集群模式下才能发挥。

14.4.3 磁盘写入与刷盘策略

当前 FileChannel.write 将数据写入操作系统页缓存即返回，并不保证立即落盘。这带来了高性能，但也意味着在掉电时可能丢失最近写入的数据。对于需要高可靠性的场景，必须提供同步刷盘选项。

优化建议：引入可配置的刷盘策略——每条消息或每 N 条消息调用一次 FileChannel.force(true)。这会导致 TPS 显著下降（可能降低 50% 以上），但能获得确定性的持久性。实际部署时，通过主从复制（集群化）通常比单机强刷盘性价比更高。

14.4.4 零拷贝与网络优化

当前消息拉取时，数据从磁盘读取到用户缓冲区，再交给 Netty 写入 Socket。这涉及两次拷贝和用户态/内核态切换。Netty 提供了 FileRegion 接口，可以直接将文件通道的内容传输到网络连接，实现零拷贝。

优化建议：在消费者拉取消息时，如果消息体较大（如超过几十 KB），可以使用 FileRegion 直接传输，绕过用户空间拷贝，显著降低 CPU 和延迟。对于小消息，零拷贝的收益不明显，可保持现有方式。

14.4.5 批量拉取与消费者效率

第八章已实现的批量拉取对消费端吞吐量帮助显著。消费者一次性拉取 50 条消息，能摊薄网络往返开销，处理效率几乎与消息处理逻辑相关。我们应在文档中指导用户，根据消息大小和处理逻辑选择合适的 maxMessages，通常 10~50 是一个不错的范围。

14.5 优化实践与最终结果

回到我们的测试环境，在 256 字节消息下，我们选择**优化序列化**作为快速验证方案。我们将消息体改为预序列化的字节数组（跳过 Jackson 的动态序列化，模拟二进制协议效果），重新测试。

- 4 线程批量发送 10 条：TPS 从 80,000 提升到 **115,000 条/秒**，成功突破 10 万大关。
- 8 线程批量发送 10 条：TPS 达到 **152,000 条/秒**，此时 CPU 使用率约 90%，磁盘写入约 38 MB/s。
- P99 延迟稳定在 30 毫秒以内，消费端无积压。

这个结果说明，**对于小消息，CPU 是首要瓶颈，通过优化序列化可以显著提升吞吐量**。如果不替换序列化方案，仅靠增加线程和批量发送，在 8~16 线程后也很难突破 10 万。而在真实生产环境中，如果使用 Protobuf 或自定义二进制格式，并结合分区和零拷贝，单机吞吐量还有进一步提升的空间。

14.6 压测中的注意事项

在进行任何性能测试时，有一些常见的陷阱需要注意：

1. **预热期**：JVM 的 JIT 编译器需要一段时间才能将热点代码编译为本地机器码。前 30 秒的数据应丢弃，测试时长至少 60 秒。
2. **客户端与 Broker 分离**：如果压测客户端和 Broker 运行在同一台机器上，客户端的 CPU 消耗会严重挤压 Broker 的资源，导致测试结果偏低。理想情况是使用独立物理机或云主机。
3. **监控资源**：测试期间必须持续监控 CPU、磁盘 IO (iostat)、网络 and GC，否则你无法知道究竟哪个环节满了。
4. **避免日志干扰**：测试时应将日志级别调整为 WARN 或 ERROR，关闭 DEBUG 和 INFO 日志，否则日志 I/O 会严重拖慢性能。
5. **文件系统选择**：Linux 下推荐使用 XFS 或 ext4，并合理设置挂载参数（如 noatime）。

14.7 本章小结

本章，我们系统地设计并执行了 min-mq 的性能测试。从单线程同步发送的 9,500 TPS，到多线程并发的 50,000 TPS，再到批量发送的 80,000 TPS，最后通过模拟二进制序列化突破 115,000 TPS，我们清晰地看到了每个优化手段的效果和代价。我们用数据证明了：**批量发送是提升吞吐量最有效的手段，序列化优化紧随其后，而磁盘 I/O 是最终无法绕过的物理极限。**

性能测试不是一次性的活动，而是一个持续迭代的过程。随着代码的修改和硬件的升级，基线数据需要不断更新。本章提供的压测框架和优化方向，可以作为你日后维护和调优 min-mq 的参考起点。现在，我们的消息中间件在功能、可靠性和性能上都达到了一个阶段性的高度。最后一章，我们将回顾整个项目，并展望从单机到集群的未来之路。

第十五章 未来之路：从原型到集群

本章前序

从第二章搭建第一个通信骨架开始，到第十四章将吞吐量推向十万 TPS，我们一行行代码地将一个想法变成了一个拥有持久化、延时投递、标签过滤、死信处理、监控告警、管理接口乃至容器化部署的完整消息中间件。min-mq 已经从“学习项目”蜕变为一个可以真正承担业务流量的单机消息系统。

但是，单个 Broker 的能力终归有限。一台服务器的磁盘容量、网络带宽、CPU 核数都存在物理上限。当业务规模继续增长，一个 Topic 的消息量超出了单机硬盘的容纳能力，或者某个消费者组的吞吐需求超过了单机网络带宽时，我们必须面对一个更加宏大的命题：如何让多台机器协同工作，组成一个逻辑上统一的、可水平扩展的消息集群？

本章不会提供具体的代码实现，而是描绘一幅通往集群世界的设计蓝图。你将看到，如何在 min-mq 现有架构的基础上，引入元数据服务器、分区、主从复制、故障转移等经典的分布式模式，最终构建出一个高可用、可扩展的消息中间件集群。这本书即将结束，但你的探索之旅才刚刚开始。

15.1 单机的极限与集群的必然

回顾我们的单机架构：所有消息都写入同一个 CommitLog（或少量分段文件），所有消费者组的索引都存储在本机文件系统，Broker 进程直接绑定特定 IP 和端口。这种设计简单、高效，在单机场景下表现得极其出色。

但它的天花板也很明显：

存储容量：一块硬盘再多也就几 TB，当消息保留时间长、写入量大时，磁盘迟早会满。

吞吐能力：即使我们优化到了十万 TPS，如果业务需要百万 TPS，单机磁盘和 CPU 就无能为力。

高可用：一旦这台机器宕机，整个消息服务完全中断，直到手动重启或更换机器。

运维弹性：面对大促流量，我们不能临时“加几台机器”分担压力；也不能在流量低谷时“撤掉几台”节约成本。

集群化的本质，就是将数据和服务分散到多个节点上，让它们像一支配合默契的乐队，既能协同放大能力，又能容忍个别成员的失误。

15.2 集群架构蓝图

一个典型的消息中间件集群包含三种角色：

Meta Server（元数据服务器）

集群的“大脑”。它存储所有 Broker 的注册信息、每个 Topic 的分区情况、每个分区的 Leader 和 Follower 分布、消费者组的订阅关系以及消费进度。通常使用 ZooKeeper、etcd 等一致性存储来保证自身高可用，或自研一个轻量级协调服务。

Broker（数据节点）

我们的 min-mq 进程的集群版本。每个 Broker 负责一部分分区（Partition）的数据存储和请求处理。它们向 Meta Server 注册自己，定期汇报心跳，接受分区的分配和迁移指令。

Client（客户端）

生产者和消费者。客户端不再直连一个固定 IP，而是从 Meta Server 获取路由表，知道“Topic A 的 Partition 0 的 Leader 在 Broker 1 上”，然后直接与相应的 Broker 通信。如果 Leader 发生切换，客户端会收到通知并更新路由。

这三者的关系可以用下图表示（图中省略了具体的箭头）：

text

[Client] ---> [Meta Server] <--- [Broker1, Broker2, Broker3]



直连 Leader Broker 进行消息收发

与单机版本相比，最重要的变化是数据不再集中在一个节点，而是通过分区打散到整个集群。

15.3 分区：伸缩性的基本单元

分区（Partition）是消息集群中最核心的概念。一个 Topic 可以被划分为多个分区，每个分区是一个有序的、不可变的消息序列，可以独立地存储在不同 Broker 上。

例如，一个 order-events Topic 可以设置 4 个分区：

Partition 0 → Broker 1

Partition 1 → Broker 2

Partition 2 → Broker 3

Partition 3 → Broker 1（一个 Broker 可以承载多个分区）

生产者发送消息时，可以根据消息中的某个键（例如订单 ID）进行哈希，将消息路由到某个固定分区，保证同一个订单的消息顺序。如果不指定键，则可以轮询或随机选择分区，达到负载均衡。

消费者组在订阅这个 Topic 时，组内的消费者会被分配不同的分区，每个分区只能被组内一个消费者消费，从而实现并行消费。当消费者组增减实例时，分区会被重新均衡（Rebalance）。

这种设计完美地将单机版本中的“一个 Topic 对应一个 CommitLog”扩展为“一个分区对应一个 CommitLog”，而每个分区的存储逻辑几乎与单机完全一致！我们的

MessageLog、ConsumeIndexManager、长轮询、标签过滤等代码，可以直接复用在分区级别上。这是多么美妙的事情——之前所有的工作都没有白费，它们只是换了一个更细粒度的上下文继续发光。

15.4 元数据服务器：集群的“大脑”

Meta Server 负责维护和协调集群状态。它的核心职责包括：

Broker 注册与发现：每个 Broker 启动时向 Meta Server 注册自己的 IP、端口、存储容量等信息，并定期发送心跳。Meta Server 据此维护一张“存活节点列表”。

Topic 与分区元数据：记录哪些 Topic 存在、各有几个分区、每个分区的 Leader 和 Follower 分别位于哪些 Broker。

消费者进度存储：将单机版本中本地文件的消费进度 (consumerOffset) 迁移到 Meta Server 的持久化存储中。这样消费者即使重启或漂移到另一台机器，也能从上次进度继续。

Leader 选举与故障转移：当 Leader Broker 宕机时，Meta Server 从该分区的 Follower 中选举一个新的 Leader，并通知所有客户端更新路由。

分区分配与再平衡：新增或下线 Broker 时，Meta Server 可以重新规划分区分布，将一些分区从负载高的节点迁移到负载低的节点。

对于我们的项目，一个简单可行的实现是引入 ZooKeeper 或 etcd 作为 Meta Server 的后端存储，然后在 Broker 端编写一个轻量的元数据管理器来与它交互。市面上许多开源消息队列（如 Kafka、RocketMQ）都采用这种做法，成熟且稳定。

15.5 数据复制与高可用

当数据只存储在一个节点上时，一旦这个节点的磁盘损坏，消息就会永久丢失。即使磁盘不坏，机器宕机也会导致服务中断。为了解决这个问题，我们需要数据复制。

每个分区可以有多个副本 (Replica)，其中一个 Leader，负责处理所有读写请求；其余是 Follower，它们被动地从 Leader 同步数据。

写入流程：生产者将消息发送给 Leader，Leader 将消息追加到本地 CommitLog 后，立即将消息推送给所有 Follower（或由 Follower 主动拉取）。当一定数量的 Follower（可配置）确认收到后，Leader 才向生产者返回 ACK。这就保证了消息在多数副本上落盘，不会因为单点故障而丢失。

故障转移：如果 Leader 宕机，Meta Server 会从存活的 Follower 中选出一个作为新 Leader。因为它拥有最新的数据，接管后可以立即继续提供服务。整个过程可能在几秒内完成，对客户端来说是透明的（客户端会收到路由更新并重连）。

我们的 ConsumeIndexManager 和 MessageLog 也可以简单扩展以支持复制：每个分区维护一个“复制偏移量”，Follower 定期告诉 Leader 自己同步到了哪里，Leader 据此判断哪些消息可以标记为“已提交”。

15.6 平滑扩容与数据迁移

集群的魅力之一，就是可以动态增加或减少节点，而不中断服务。

扩容：当新的 Broker 加入集群时，它一开始没有任何分区。管理员可以通过管理命令或自动均衡器，将某些 Topic 的某些分区迁移到新 Broker 上。迁移过程通常是：先在新 Broker 上创建一个 Follower 副本，让它从 Leader 同步数据；同步完成后，进行一次 Leader 切换，将新 Broker 上的副本提升为 Leader；然后删除旧节点上的副本。整个过程对生产者只造成短暂的阻塞（Leader 切换瞬间），而消费者可以无缝重连。

缩容：需要下线一台 Broker 时，先将该 Broker 上的所有分区安全地迁移到其他节点，确认所有分区都已迁出后，再从集群中注销该 Broker。最后才能安全关机。

这些工具（分区重新分配、Leader 切换）在 Kafka 和 RocketMQ 中都有成熟实现，

其原理清晰，可以作为我们集群化改造时的参考。

15.7 客户端的自适应进化

集群化后，客户端也需要变得更加智能。它不再只是一个“连某个固定地址，发消息、收消息”的简单程序，而必须能：

动态路由发现：启动时从 Meta Server 拉取 Topic 的分区元数据，并缓存。在连接断开或收到路由更新通知时，重建路由表。

连接池管理：与多个 Broker 建立连接（每个分区的 Leader 至少一条连接），而非全局一条连接。

分区选择策略：生产者支持按 key 哈希、轮询或随机选择目标分区；消费者组内进行分区分配，支持粘性分配或轮询分配。

故障重连与 Failover：当某个 Broker 连接断开时，自动切换到该分区的其他副本（如果知道的话），并从 Meta Server 拉取最新路由。

这些能力在很大程度上可以基于我们第十一章实现的断线重连机制进行扩展。单机重连是“同一个地址多试几次”，而集群重连是“换个新地址再试”。

15.8 性能、可靠性与复杂性的权衡

分布式系统永远是一门权衡的艺术。集群化带来了弹性扩展和高可用，但也引入了新的挑战：

网络开销：复制会消耗额外的网络带宽，Leader 和 Follower 之间的同步可能导致写入延迟上升。

一致性 vs 可用性：在发生网络分区时，是优先保证数据一致性（牺牲部分可用性），

还是优先保证可用性（可能丢失部分数据）？这是著名的 CAP 定理，需要根据业务场景做出选择。

运维复杂度：集群需要监控、自动修复、滚动升级等运维工具，这些都增加了系统的整体复杂度。

对于我们的项目，一个务实的做法是渐进式集群化。第一步，可以先实现“静态分区”——通过配置文件指定 Broker 和分区的对应关系，由客户端直接读取，不引入 Meta Server。第二步，引入 Meta Server（如 ZooKeeper），将元数据集中管理。第三步，实现主从复制和故障转移。每走一步，都用本书提供的测试手段验证其正确性和性能。最终你就能得到一个麻雀虽小、五脏俱全的分布式消息集群。

15.9 全书总结与寄语

亲爱的读者，我们共同走过了十五个章节。你从一开始搭建 Maven 项目、编写第一个 Netty 通信骨架，到逐步添加持久化、延时消息、标签过滤、监控、批量处理、死信队列、优雅关闭、断线重连、按时间消费、管理 API，再到压测调优和集群蓝图——你亲手建造了一个消息中间件，并在这个过程中深入理解了它的每一个细胞。

但这仅仅是开始。你可能会继续为 min-mq 添加事务消息（与本地数据库事务绑定）、实现基于 TLS 的安全通信、设计一个漂亮的 Web 管理控制台、或者干脆用 Go 语言重新实现一套高性能版本。你也可能将在这里学到的设计思路——顺序写磁盘、时间轮、长轮询、分段清理——应用到你自己的项目中去。

技术之路，最宝贵的不是某个特定的工具或框架，而是通过亲手“造轮子”而获得的深刻理解，以及在面对未知时敢于动手的勇气。消息中间件的世界远比本书所覆盖的更广阔，但只要掌握了解构复杂系统的能力，任何看似遥不可及的目标，都可以一步步拆解、实现。

愿你永远保持这份热情与好奇心，继续在技术的海洋中乘风破浪。我们江湖再见。

15.10 本章小结

本章，我们站在单机 min-mq 的肩膀上，眺望了集群化的远方。我们讨论了 Meta Server、分区、主从复制、Leader 选举、数据迁移和客户端自适应等核心概念，勾勒出一条从单机到集群的渐进式演进路径。集群化不是推翻重来，而是在现有代码基础上的自然延伸——分区内的存储逻辑与单机几乎完全相同，这正是好的架构设计所带来的扩展力。

全书至此完结。min-mq 已经从一个想法，变成了一套可以运行、可以压测、可以监控、可以部署的消息系统，更重要的是，你已经拥有了一把将复杂系统化整为零的“手术刀”。接下来，轮到你去创造新的可能了。